



SDVoE Solution

Introduction to the SDVoE API User Guide

This page is intentionally blank.

Semtech Confidential Information
Mark Li
ADV
02/03/2026

Contents

1	PURPOSE.....	5
1.1	REFERENCE MATERIALS.....	5
2	INTRODUCTION TO THE SDVOE API.....	7
2.1	ARCHITECTURAL OVERVIEW	7
2.1.1	<i>The REST API</i>	7
2.1.2	<i>WebSocket</i>	8
2.1.3	<i>Application Architecture</i>	8
2.1.4	<i>Relevant Specifications</i>	10
2.2	API VERSIONING AND APPLICATION COMPATIBILITY	10
2.2.1	<i>API Version Numbering Scheme</i>	10
2.2.2	<i>Require API Version</i>	11
2.3	EVENTS.....	11
2.3.1	<i>Event Types</i>	12
2.3.2	<i>Tracking Device State</i>	12
2.3.3	<i>Event Buffering</i>	13
2.3.4	<i>Polling Events</i>	13
2.3.5	<i>Handling Asynchronous Events</i>	15
2.4	COMMANDS TYPES	16
2.4.1	<i>Background Commands</i>	16
2.4.2	<i>Immediate Commands</i>	19
2.5	DEVICE MODEL.....	19
2.5.1	<i>Device Life Cycle</i>	20
2.6	OVERVIEW OF THE API COMMAND STRUCTURE	21
2.6.1	<i>Error objects</i>	22
2.6.2	<i>HTTP Status</i>	24
2.6.3	<i>Device Error Objects</i>	25
2.7	DESCRIPTION OF STREAMS AND DESTINATIONS.....	26
2.7.1	<i>Joinable streams</i>	26
2.7.2	<i>Switchable streams</i>	27
2.8	MULTICAST IP ADDRESS MANAGEMENT.....	27
2.8.1	<i>Multicast Usage</i>	27
2.8.2	<i>Possible Multicast Address Conflicts</i>	28
3	HELPFUL GUIDELINES.....	29
3.1	<i>“GET SETTINGS” CAN RETURN PROCESSING STATUS</i>	29
3.2	<i>APPLYING VIDEO COMPRESSION TO HDMI NATIVE STREAM</i>	29
3.3	<i>REMOVING HDMI AUDIO FROM HDMI VIDEO STREAM</i>	30
3.4	<i>GUIDELINES FOR USE OF THE SDVOE API</i>	31
3.4.1	<i>Checking for Supported Features</i>	31
3.4.2	<i>Monitoring Event Notifications</i>	32
3.4.3	<i>Establishing Communication between a Client and the API</i>	32
3.5	<i>ENABLING AND DISABLING THE FRAME RATE CONVERTER</i>	34

4	CORE COMMANDS	35
5	MANAGING THE VIDEO OUTPUT FORMAT	37
5.1	SPECIFYING A VIDEO OUTPUT FORMAT	37
5.1.1	Video Display Modes.....	37
5.1.2	Video Format Arguments	38
5.1.3	Uniquely Identifying a Video Format	43
6	BASIC API COMMANDS.....	45
7	DEVICE DISCOVERY AND FEATURE DETECTION	49
8	REVISION HISTORY	100

List of Figures

Figure 1:	Example of a simple API-based architecture	9
Figure 2:	Example of an API design incorporating a Manager/Proxy (Web Server)	9
Figure 3:	Lifecycle of a Background command	17
Figure 4:	API Device life cycle	21
Figure 5:	Illustration of Grid Indexes for a 3x2 Video Wall	42

List of Tables

Table 1:	RFC Specification Links	10
Table 2:	SDVoE API device model – Components of Device Objects	20
Table 3:	API JSON responses.....	22
Table 4:	API error objects.....	22
Table 5:	Device API error objects.....	24
Table 6:	Relation between error reasons and HTTP status codes	25
Table 7:	Device API error objects.....	26
Table 8:	API Command Modes.....	35
Table 9:	Version command return values	36
Table 10:	Version JSON Module description Object.....	36
Table 11:	API Video Format Arguments.....	39
Table 12:	Supported Video Identification Codes (VIC)	42
Table 13:	List of API commands	45

1 Purpose

This document introduces users to the concepts and architecture of the SDVoE API that need to be considered when developing client applications. It provides guidelines and information that is required when designing applications that will communicate and manage endpoint devices through the API.

Applies to a wide range of chipsets, including:

- BlueRiver AVP1000/AVP2000/AVP2000T chipsets
- BlueRiver NT1000/NT2000 chipsets
- USB 2.0 extender UEX1000/UEX2000 chipsets

This document applies to the SDVoE API version 3.9.0.0.

The target group is software designers/developers, project managers, support teams and others involved in the evaluation, design, coding or deployment of client applications that are to manage SDVoE devices implementing the BlueRiver professional AV technology.

1.1 Reference Materials

The supported API command set is reviewed in detail in the SDVoE API Developers Reference Guide (PDS-062489), including command syntax, arguments and examples. The SDVoE Control Server Administrators Guide (PDS-062492) provides information related to the API Server, including details on port usage and key items that may be configured.

In addition, the SDVoE API Application Note Library (PDS-062491) is useful in assisting with the development of client applications that are to interact with the SDVoE API. It includes details on a series of topics including multiview mode, wall mode, quantization and examples of use cases for commonly managed property nodes.

This page is intentionally blank.

Semtech Confidential Information
Mark Li
ADV
02/03/2026

2 Introduction to the SDVoE API

The SDVoE Application Programming Interface (API) is designed to allow for customer applications to manage and communicate with devices implementing BlueRiver and/or UEX chipsets. It provides an extensive range of commands and arguments to provide a powerful and flexible solution for professional AV products.

Prior to proceeding with a project it is first necessary to obtain a basic understanding of the API itself. It is necessary to plan the actions that each client needs to perform to communicate, manage and initiate a session with the SDVoE Control Server (API), hereafter referred to as the API server.

2.1 Architectural Overview

The following provides an architectural overview regarding the API.

- The API server is a multi-platform software application that allows client software to configure and manage devices and other components of a system. It uses a Client-Server Architecture where the API server functions as the server and the application software acts as the client.
- API server allows multiple clients to interact with it concurrently.
- Clients manage the system by sending requests to the server.
- They then receive responses back from the server, as well as event notifications.
- Three communication interfaces are provided for this purpose:
 - A REST API, where the client interacts with the server through HTTP requests.
 - WebSocket connections that allow clients, including in-browser web applications, to receive asynchronous event notifications.
- Each response or event notification from the server is a single JSON object. A common structure is applied to all responses and event notifications.

2.1.1 The REST API

SDVoE Control Server supports a REST API over HTTP protocol version 1.1:

- Clients perform HTTP requests with the `GET`, `POST` or `DELETE` method on various resources.
- Arguments for `POST` requests are provided in a JSON object sent as the request body. Typically `GET` requests do not require arguments; for those that do, these arguments are passed as the query part of the request URI. `DELETE` requests require no argument.
- API responses are provided as the HTTP response body.
- HTTPS is supported.
- The TCP port is configurable. The default port number is 80 for the REST API over HTTP and 443 for the REST API over HTTPS.

The recommended media type for the request body of a POST request is `application/json`.

Some additional media types are recognized for interoperability. These are as follows:

- `application/json` (standard, recommended)
- `text/json`
- `text/x-json`
- `text/javascript`

No other media type is supported.

API responses are always served with media type `application/json`.

The REST API supports all features of the API except for asynchronous (i.e. server push) event notifications. This is because they do not map well on the request/response nature of HTTP. A client that needs to handle these notifications can do so using a WebSocket connection.

2.1.2 WebSocket

The REST API supports WebSocket connections. The main envisioned use case is to serve as a delivery mechanism for asynchronous event notifications from the API.

- As per the WebSocket standard (RFC 6455), a WebSocket connection is established by performing an HTTP protocol upgrade request on an HTTP resource. In the case of this API, the resource to establish this connection is `/api`.
- The API uses WebSocket message framing: each response or command is a message. Carriage return and line feed characters are not significant for framing and are considered whitespace.
- Asynchronous event notifications are enabled on a newly established WebSocket connection.
- Except for the above or as otherwise noted, following also applies to the WebSocket:
 - ⊖ A client using a WebSocket connection solely to receive event notifications, which is the main intended use case, does not need to take any actions or send any commands to enable notifications or otherwise set up the connection.

2.1.3 Application Architecture

This sections provides examples of architectures that incorporate the SDVoE API.

The first example shown is a simple architecture where a single client communicates directly with the API server. This single client could be:

- A Graphical User Interface (GUI) implemented as a desktop application.
- A GUI that runs as a Single Page Application (SPA) in a web browser.
- An application without a graphical interface that controls the system automatically according to internal rules.

In this architecture example, the client communicates with the server through the REST API and a WebSocket connection. The WebSocket connection is optional so is shown in dashed lines but as mentioned previously it is recommended is as used to handle event notifications.

Only one client is shown in this example but the API server fully supports multiple clients.

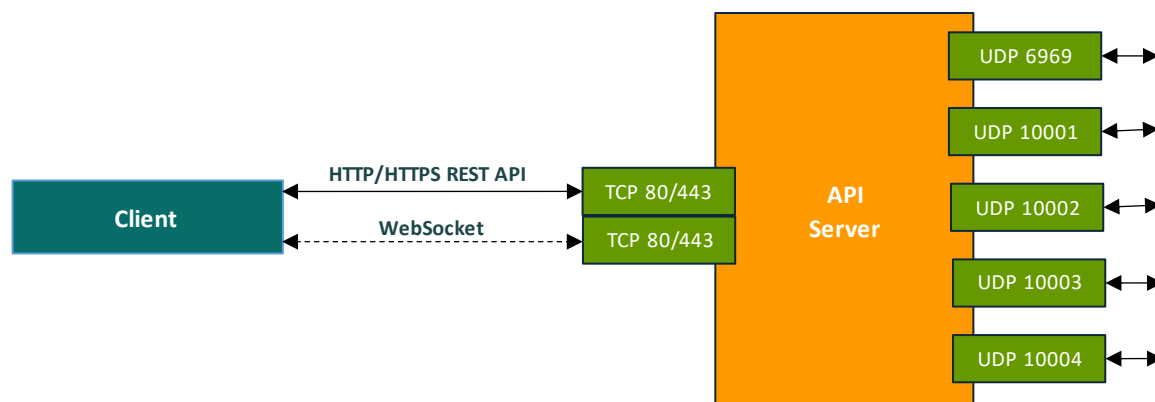


Figure 1: Example of a simple API-based architecture

The next example shows a more complex architecture that interposes an intermediate manager or proxy application between the API server and clients. Motivations for such an architecture include:

- Provide a layer to abstract the differences between SDVoE endpoint devices and other kinds of endpoints or equipment that also need to be controlled.
- The operation/client software is running on a dedicated third-party AV control system or mobile device.

It is possible to have the API server and the manager/proxy application running on the same machine. When this is done, the API server can be configured to allow only connections from software running on the local machine.

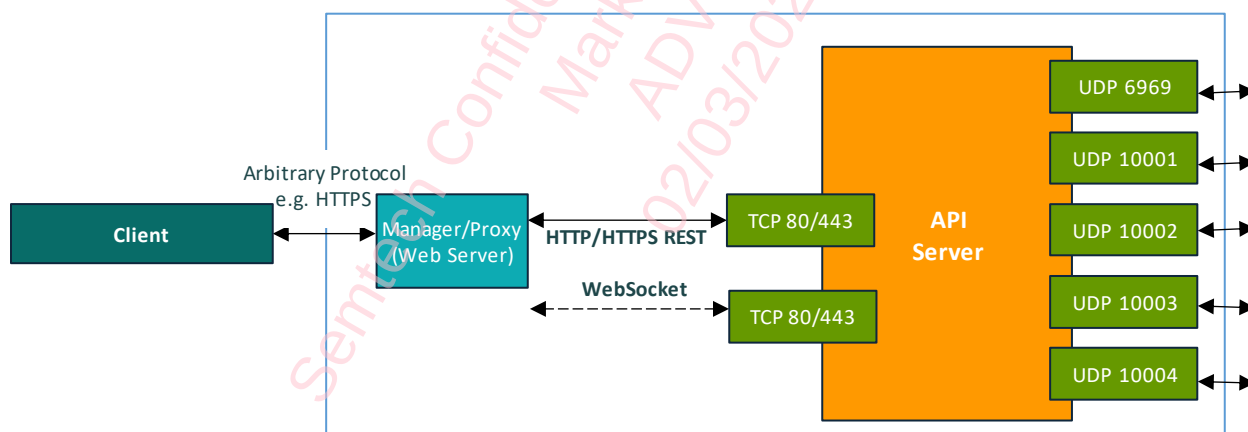


Figure 2: Example of an API design incorporating a Manager/Proxy (Web Server)

2.1.4 Relevant Specifications

Refer to the following RFC specifications for more information on relevant data formats and protocols. The table below provides some RFC links.

Table 1: RFC Specification Links

Title	Internet Engineering Task Force (IETF) links
Telnet Protocol Specification	http://tools.ietf.org/html/rfc854
Telnet Option Specification	http://tools.ietf.org/html/rfc855
The WebSocket Protocol	http://tools.ietf.org/html/rfc6455
The JavaScript Object Notation (JSON) Data Interchange Format	http://tools.ietf.org/html/rfc7159
Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing	http://tools.ietf.org/html/rfc7230
Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content	http://tools.ietf.org/html/rfc7231
Hypertext Transfer Protocol (HTTP/1.1): Caching	http://tools.ietf.org/html/rfc7234

2.2 API Versioning and Application Compatibility

2.2.1 API Version Numbering Scheme

The API version uses a version numbering scheme expressing compatibility information. The complete version numbering has the following format `major.minor.revision.hotfix`.

As an example, for hypothetical API version 3.4.5.6, the major number is 3, the minor number is 4, the revision number is 5 and the hotfix number is 6.

Versions with the same `major` and `minor` numbers, differing only in the revision and/or hotfix number are `forward` and `backwards` compatible with each other. For example, if an application is developed for hypothetical API version 3.4.5.6, then the application is able to interact without issue with API versions 3.4.3.0 (lower revision), 3.4.6.0 (higher revision), 3.4.5.5 (lower hotfix) or 3.4.5.7 (higher hotfix).

Revisions and hotfixes introduce fixes or internal improvements that do not affect the programming interface. For this reason, this document refers to API versions only by their `major` and `minor` number. For example, API 3.0.

Versions that differ in their `minor` number are generally `backwards` compatible but are not forward compatible. This is because versions with a new minor number introduce new requests or optional arguments, new JSON members or other functionality that was not present in earlier versions. Refer to the appropriate release notes for more details on changes applied in a new release of the API.

For example, an application developed for hypothetical API version 3.4.5.6 will run “as-is” on API version 3.5.0.0 (higher minor) without problem. However, an application developed for 3.4.5.6 may be using features not present in 3.0.0.0 and as such will not be able to successfully run on API version 3.0.0.0.

When versions differ by their `major` number, effort is required to port the application from one version to another. For example, an application developed for the API version 2.21.0.0 will not work “as-is” with API version 3.0.0.0.

To ensure compatibility of your application with future API versions, it is recommended to follow the following rules:

- i. As new API versions add new members to the JSON response returned by commands, it is important that the application ignore members it does not know about.
- ii. As new API versions add new event types, it is important that the application ignore events it does not know about. Refer to the description of the `event` command in section 6 Basic API Commands.
- iii. As new API versions can add new nodes, subscriptions and stream types, it is important that the application ignore node, subscription and stream types it does not know about. Additional information on device objects is provided in the SDVoE API Developers Reference Guide.

2.2.2 Require API Version

The first thing an application is expected to do when it starts interacting with the API, is to declare the API version it is expecting. This is highly recommended for the REST API and mandatory on WebSocket connections if they are used to issue commands and not to simply handle event notifications.

The application declares its expected version of the API by using the `require` command (WebSocket) or its equivalent REST API request. As part of this command or request, the application specifies the actual version of the API for which it was developed. The `require` command checks the provided version with the current API version according to the rules described above to ensure all the required features are present. Example of `require` request and its return is provided in section 3.4.1 Checking for Supported Features.

2.3 Events

Interesting occurrences, such as a change to a device's state or the reception of RS-232 data from a device, trigger an event. Handling these "events" allows client applications to remain up to date with the system's state.

Each event has an `event type` and most event types have associated data. Events that contain data have an associated `request ID`. This unique ID is then sent using the `request` command to retrieve data associated to the event.

Such events can be managed either by:

1. Enabling `asynchronous event notifications` to automatically trigger notifications when new events arrive.

Note: This is the recommended approach.

The following guidelines apply:

- Asynchronous mode requires use a WebSocket connection.
 - On a WebSocket connection, notifications are enabled by default.
 - Refer to section 6 Basic API Commands for an overview of the asynchronous mode command and to the SDVoE API Developers Reference Guide for more detailed information on this command.
2. Alternatively, it is possible to `poll` the list of latest events. This is done by using the `event` command.

2.3.1 Event Types

Events that are triggered by the API Server include the following:

DEVICE_CONNECTED	A new device had been discovered.
DEVICE_DISCONNECTED	Device has disappeared from the network.
DEVICE_INFO_AVAILABLE	Information about a new device is available.
REQUEST_COMPLETE	A request command has completed.
SETTINGS_CHANGED	A change has occurred to a device's settings.
MULTIVIEW_CHANGED	Layout loaded on a receiver (RX) device has changed.
I2C	Indicates that an I2C bus master, such as the external MCU, generated an event through the I2C interface.
CEC_RECEIVED	Consumer Electronics Control (CEC) data received from a device.
RS232_RECEIVED	RS-232 data received from a device.
INFRARED_RECEIVED	Infrared data received from a device.
USB_TREE_CHANGED	The tree of downstream connected USB devices changed. Applies to UEX, in cas of UEX attached to a BlueRiver AVP device there will be 2 events (one from the UEX device and one from the AVP).
FIRMWARE_UPDATE_START	A device's firmware has begun an update.
FIRMWARE_UPDATE_PROGRESS	Indicates the progress of a device update.
FIRMWARE_UPDATE_COMPLETE	A device's update has completed.

2.3.2 Tracking Device State

The following event types provide information about change in a device's state, including whether it is present or not on the network, its status and its configuration:

DEVICE_CONNECTED
DEVICE_DISCONNECTED
DEVICE_INFO_AVAILABLE
SETTINGS_CHANGED
MULTIVIEW_CHANGED

Furthermore, these event types provide information about a device's firmware update status:

FIRMWARE_UPDATE_START
FIRMWARE_UPDATE_COMPLETE
FIRMWARE_UPDATE_PROGRESS

Tracking a device's current state can be done by handling these events. Handling all events of the above types for a specific device starting from the start of the event timeline always yields the correct state and firmware update status. The same is true if events are handled incrementally: starting from a specific point in the event timeline and a state that was known to be correct at that point, handling all

events that occurred since that point yields the correct current state.

Note: Tracking devices solely using `get settings` or `get device` commands is the not recommended method and is discouraged for operations with more than a handful of devices. Semtech recommends to implement asynchronous event handling.

2.3.3 Event Buffering

The API Server tracks events buffering the data returned from transmitter (TX) and receiver (RX) devices as outlined below.

For the event types listed below, related data is buffered by the API Server for 10 minutes, after which each event and associated data is discarded:

```
REQUEST_COMPLETE
MULTIVIEW_CHANGED
I2C
INFRARED_RECEIVED
RS232_RECEIVED
CEC_RECEIVED
```

All other events are guaranteed to persist for at least 30 seconds.

Furthermore, as indicated in the previous section, it is possible to track the current state and firmware update status of a device by handling events. This means all events that are needed to reconstruct the current state of a device are kept around for at least as long as they are needed for that purpose. Events that no longer contribute to the current state because their contribution is invalidated or made redundant by more recent events may be discarded.

2.3.4 Polling Events

Periodically polling for device statuses is relatively efficient for a small number of devices (100-200). However, it needs to be considered that the number of commands and the volume of data exchanged quickly increases as the system (number of devices) grows. This can impact the response time. The concern is less about scalability than it is about latency (responsiveness) as the device count increases.

When a new device connects to the API Server, a series of events trigger to update all the clients with the current state of all known devices.

The following events may be raised:

<code>DEVICE_CONNECTED</code>	Event for each device present on the network.
<code>DEVICE_INFO_AVAILABLE</code>	Event follows shortly the <code>DEVICE_CONNECTED</code> event with detailed information on the device.
<code>DEVICE_DISCONNECTED</code>	Event for each device previously seen by API Server but currently not present on the network.
<code>FIRMWARE_UPDATE_START</code>	Event for device that is currently updating.

To poll device events, the client software needs to complete the following:

1. Retrieve events from the start of the event timeline, call the `event` command without specifying an event ID.

```
GET /api/event
```

Optionally, you can use the `limit` query argument to limit the number of events returned by the API, as follow (10 events in this example):

```
GET /api/event?limit=10
```

2. The API Server returns a list of all the events that have occurred, or up to the limit specified if applicable.
3. The client application needs to proceed to process each event retrieved in the list.
4. Next using the `request` command, along with the appropriate `request_id`, retrieve the data associated with each event of interest.

Note: Each event reported includes a unique `request_id`.

5. In addition to the event type and request ID, each event also contains an event ID. To retrieve the next events, if any, call the `event` command again, this time passing the event ID of the last event you retrieved as the `after` query string argument.

Below is an example of an `event` command, using an event ID of 49 passed in the `after` query string argument and a limit of 10 events:

```
GET /api/event?after=49&limit=10 HTTP/1.1
...
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "events" : [{
      "device_id" : "d8803962abcd",
      "event_id" : 50,
      "event_type" : "SETTINGS_CHANGED",
      "timestamp" : 1450386633,
      "request_id" : 29
    },
    {
      "device_id" : "d8803962b2d9",
      "event_id" : 51,
      "event_type" : "DEVICE_CONNECTED",
      "timestamp" : 1450386633,
      "request_id" : null
    },
    {
      "device_id" : "d8803962b2d9",
      "event_id" : 52,
      "event_type" : "DEVICE_INFO_AVAILABLE",
      "timestamp" : 1450386633,
      "request_id" : 30
    }
  ]
}
```

```
    },  
    "error" : null  
}
```

2.3.5 Handling Asynchronous Events

The asynchronous event mode is the recommended approach, particularly for systems with a large number of end-points or that are expected to potentially grow. The API Server sends notifications to an active client when a new event occurs. This removes the need to poll events using the event command.

Notifications are JSON objects that have the same format as an API response, except that the status member is `NOTIFICATION`. The results member of an asynchronous notification has the same format as the return value of an event command.

Because of the request-response nature of the HTTP protocol, the REST API cannot be used to handle asynchronous event notifications. Instead, a WebSocket connection must be used for this purpose:

- Connect the WebSocket to the `/api` URI.
- Asynchronous notifications are enabled by default on a WebSocket connection. Each event notification is a WebSocket text message.

Although the asynchronous method offers the best responsiveness (as there is no polling delay), handling events asynchronously is programmatically more challenging because event notifications are interspersed with command responses. There is however, a suggested way to achieve this in a structured and reliable manner. The following section provides a sample approach.

2.3.5.1 Handling Events Asynchronously

Handling events asynchronously can be managed as outlined below, however should be noted that this is only one of multiple potential approaches:

1. When a command is sent to the API Server, block until a response is received. Since responses are received asynchronously by another thread, a shared variable or a semaphore could be used to unblock the sending thread. The send function will then return.

After sending a command, a response is always received with a status of `SUCCESS`, `ERROR` or `PROCESSING`.

2. Read responses and notifications asynchronously in a thread.

It is necessary to read both command responses and asynchronous event notifications since they arrive through the same connection.

The action to be taken for each response is dependent on the response's status member.

- If the status is `SUCCESS` or `ERROR`, the pending request is completed.
- If the status is `PROCESSING`, the request ID is recorded and the request is added to a processing list.
- The sending thread can be unblocked since a response was received.

An asynchronous notification is received when the status is `NOTIFICATION`. A notification will contain a list of events. For each event:

- If the event type is `REQUEST_COMPLETE`, the request referenced by the request ID is completed and can be removed from the processing list. A callback mechanism can be used to update the client interface.

- If the event type is anything other than `REQUEST_COMPLETE`, handle the event according to the application logic.

2.4 Commands types

The API Server supports two types of commands:

- Immediate:** This type of command always immediately returns with either a `success` or `error` status.
- Background:** These commands complete a task in the background. The response of these commands is retrieved using a `request` command. As mentioned above, a unique request ID is assigned to each command of this type when it is issued.

2.4.1 Background Commands

Implementing `background` commands prevents stalling client software execution. They prevent the API from waiting for a device to complete a command's execution. Instead, commands return with a status of "PROCESSING" along with a unique request ID in the `request_id` member of a response.

The `request` command is then used with the request ID to retrieve the command's return.

In the REST API, a `PROCESSING` response has the HTTP status 201 Created. The `Location` HTTP header of the response provides a link to the appropriate request resource.

The API Server buffers command returns and notification events. Refer to section 2.3.3 Event Buffering for more information.

The lifecycle of a background command is as follows:

- Valid background commands return with the status member set to `PROCESSING`, along with a unique request ID specified in the JSON response.
- In case of an error, such as "illegal argument", the command immediately returns an `ERROR` status.

TIP: For more information on error objects refer to section 2.6.1 Error objects.

- After a command has been issued, its status is retrieved using the request ID through the `request` command:
 - If the command is still being handled, the `request` command returns a `PROCESSING` status.
 - Once the command has completed, the `request` command returns a `SUCCESS` status, as well as the return value (i.e. returned data) of the command.
- Once a command completes, the event `REQUEST_COMPLETE` is triggered.
 - Once a `REQUEST_COMPLETE` event is triggered for a given request ID, the `request` command is guaranteed to return with a `SUCCESS` status when the request ID is called.

TIP: For information on the `request` command refer to section 6 Basic API Commands.

Summary of Command Lifecycle

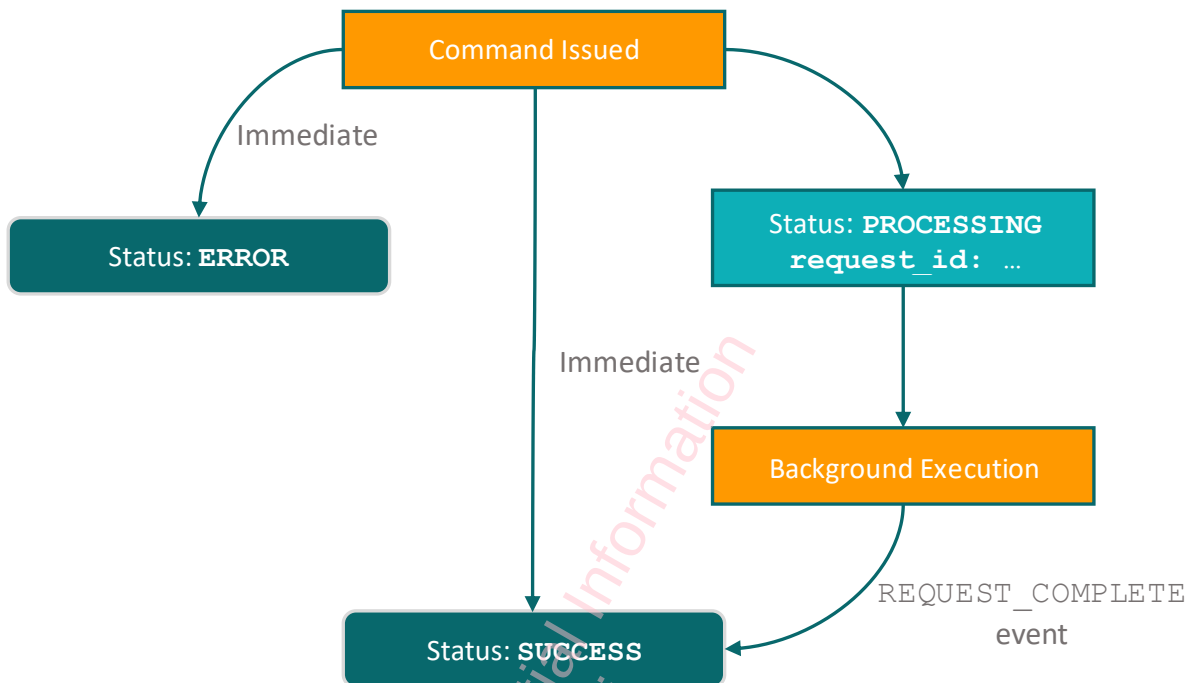


Figure 3: Lifecycle of a Background command

2.4.1.1 Monitoring Command Execution

Command syntax: `GET /api/request/{request_id}`

Example of the `request` command.

Reminders:

- The `request` command retrieves either the return value of a background command, that is a command that responded with a `PROCESSING` status and a request ID or the data associated with an event.
- The `request_id` is a request identification number included in either in the `PROCESSING` command response or the associated event object.
- The API Server buffers command returns and notification events. Refer to section 2.3.3 Event Buffering for more information.
- Refer to the SDVoE API Developers Reference Guide for details on API command syntaxes and usage.

The following example illustrates confirming the successful execution of a `stop` command sent to a specific transmitter device.

```
POST /api/device/001ec0f03668 HTTP/1.1
...
{
  "op" : "stop",
  "stream_type" : "HDMI",
  "stream_index" : 0
}
```

```
HTTP/1.1 201 Created
Location: /api/request/85521
...
{
  "status" : "PROCESSING",
  "request_id" : 85521,
  "result" : null,
  "error" : null
}
```

Response from the API Server for the request command associated the request ID associated of the stop command, note status is "PROCESSING":

```
GET /api/request/85521 HTTP/1.1
...
HTTP/1.1 200 OK
{
  "status" : "PROCESSING",
  "request_id" : 85521,
  "result" : null,
  "error" : null
}
```

Response example of the API Server for the "REQUEST_COMPLETE" event raised when stop command has completed:

```
GET /api/event?after=85528 HTTP/1.1
...
HTTP/1.1 200 OK
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "events" : [
      {
        "device_id" : null,
        "event_id" : 85529,
        "event_type" : "REQUEST_COMPLETE",
        "timestamp" : 1462902262,
        "request_id" : 85521
      }
    ],
  },
  "error" : null
}
```

```

GET /api/request/85521 HTTP/1.1
...
HTTP/1.1 200 OK
{
  "status" : "SUCCESS",
  "request_id" : 85521,
  "result" : {
    "devices" : [
      {
        "device_id" : "001ec0f03668",
        ...
      }
    ],
    "error" : []
  },
  "error" : null
}

```

For more details on events refer to section 2.3 Events and for information on the `get` command refer to section 6 Basic API Commands.

2.4.2 Immediate Commands

Unlike background commands, immediate commands return a response straightaway.

API immediate commands include the following:

<code>event</code>	
<code>layout</code>	Applies to multiview only.
<code>list</code>	
<code>mode</code>	
<code>quit</code>	
<code>require</code>	
<code>version</code>	
<code>request</code>	Retrieves return after REQUEST_COMPLETE event occurs.

2.5 Device Model

The heart of the API is its “device model”.

The API device model describes each device’s status, configuration, data connectivity, as well as supported capabilities. It is represented with a JSON object referred to as “Device Objects”.

A device object is returned as the result of some commands.

- Notably this occurs with a `get` command, such as `get device` or `get settings`.
- As well a device object is returned by the `request` command when it is called with the request ID of specific event types, including `DEVICE_INFO_AVAILABLE`, `SETTINGS_CHANGED` and `REQUEST_COMPLETE`.

The following table outlines the components that compose a device object.

Table 2: SDVoE API device model – Components of Device Objects

Device Objects Component	Description
Device Identity	Includes information about a device, such as device type and firmware version.
Device Configuration	Contains device settings. These are configured through either API commands and/or applied during initial firmware programming.
Device Status	Includes information pertaining to current device status. Example, if the device is active and connected to network switch or functioning in point-to-point mode.
Device Streams	Describes all data streams sent out from a transmitter device.
Device Subscription	Describes the data streams that a receiver is currently subscribed to.
Device Nodes	List/describes present hardware modules along with their status and configuration. Includes HDMI I/O, Audio I/O, Network interface, Frame Buffer, Scaler, etc.

Additional information on Device Object Definitions refer to the SDVoE API Developers Reference.

2.5.1 Device Life Cycle

When a device is discovered on the network until it's disconnected, events are triggered that client applications can then retrieve. Tracking events allows clients to track each device and to receive notifications from registered devices.

Specifically, client applications can send commands to a device after a `DEVICE_INFO_AVAILABLE` event has been received and prior to a `DEVICE_DISCONNECTED` event occurs. Between these events is also when non-life cycle-related events are received.

For example, if a device unexpectedly disappears, this could be indicating a device or network failure.

2.5.1.1 Overview of device life cycle:

When a new device is discovered on the network, the event `DEVICE_CONNECTED` occurs.

- Device is seen but no further communication established with device.
- Device considered active, `status.active` becomes `true` in the device model.

The `DEVICE_INFO_AVAILABLE` event occurs after the `DEVICE_CONNECTED` occurred, once the API Server can retrieve all information from the device. This signals that the device is accessible and can be controlled.

While the device is active, it can trigger the `SETTINGS_CHANGED` event, indicating that the state of the device has changed. For example, a state change that triggers a `SETTINGS_CHANGED` event is a video source resolution change or a monitor that is connected to or disconnected from a receiver.

When a device disappears from the network, the `DEVICE_DISCONNECTED` event is triggered. The device is still listed in the device list that is retrieved with the `get` command, but it is shown to be inactive (i.e. `status.active` becomes `false` in the device model).

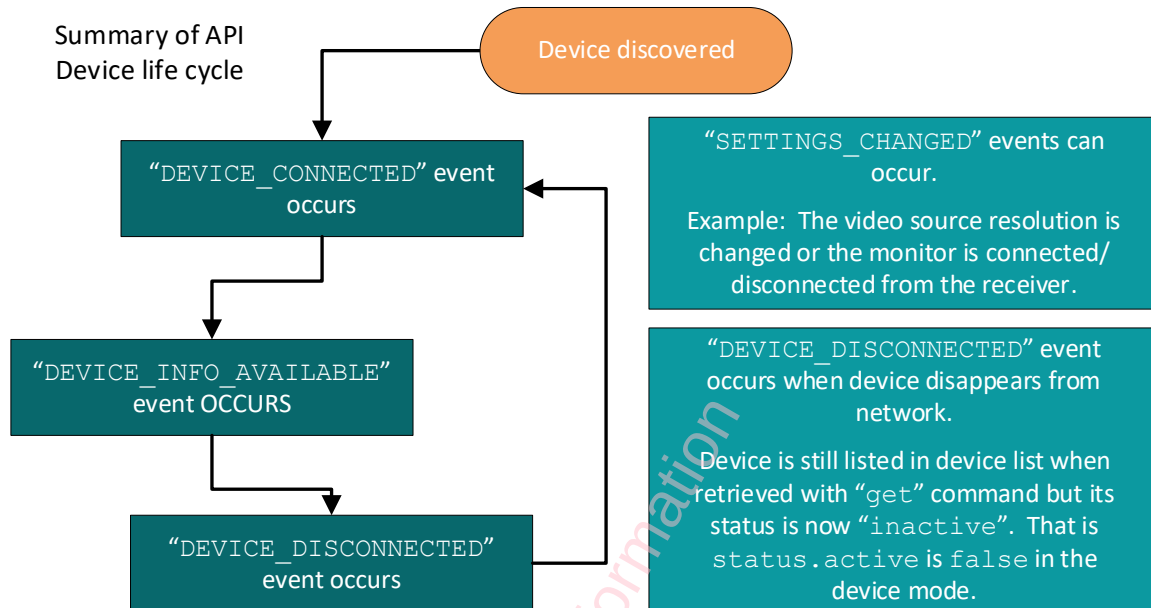


Figure 4: API Device life cycle

Below is an example of a snippet taken from the response of a `get` command showing the status of an active device (`true`):

```

},
"status" : {
  "active" : true,
  "point_to_point" : false,
  "temperature" : 78
},

```

2.6 Overview of the API command structure

All API commands generate a JSON response similar to the following:

```

{
  "status" : String,
  "request_id" : Integer,
  "result" : Varies,
  "error" : Error
}

```

Table 3: API JSON responses provides descriptions for the JSON responses that may be received for API request.

Table 3: API JSON responses

Member name	Since	Type	Description
status	2.0	String	Possible results for <code>status</code> are: SUCCESS: Command succeeded. ERROR: Command failed. PROCESSING: Command executed as a background command. Possible to execute other commands while waiting for result. Refer to <code>request</code> command for details on retrieving return value once command completed. Refer to <code>event</code> command and <code>REQUEST_COMPLETE</code> event type for details on setting notifications when command completes. In addition, the <code>NOTIFICATION</code> status is never encountered in a command response but indicates an asynchronous event notification when feature is enabled. Refer to description of the mode <code>async</code> command for details.
request_id	2.0	Integer	A unique <code>request</code> ID is assigned to each command executed as a background command. This member is <code>null</code> otherwise.
result	2.0	Varies	Result return value of a command. The result is set when the status reported is <code>SUCCESS</code> , otherwise <code>null</code> is reported. Return values are documented in reference section of each command in the API Developers Command Guide.
error	2.0	Error	Error object describing reason why a command was unsuccessful. Set status is <code>ERROR</code> , otherwise <code>null</code> is reported.

2.6.1 Error objects

An error object has following structure:

```
{
  "reason": String,
  "message": String
}
```

Table 4: API error objects

Member	Since	Type	Description
Reason	2.0	String	A machine-readable strings identifying the error reported: AUTHENTICATION: Operation failed because the client was not authenticated. Note: TLS client certificate is currently the only supported means of authentication. ILLEGAL_ARGUMENT**: Invalid argument specified for command. INVALID_STATE**: Operation cannot be performed on the device in its current state, such as when a firmware update is in progress.

Member	Since	Type	Description
			Tip: Same command may succeed later if condition resolved. INVALID_COMMAND**: Entered command does not exist. Verify the appropriate API module loaded. INVALID_STATE: Operation cannot be performed on device in its current state. For example a firmware update is in progress. Tip: Same command may succeed later if condition resolved. IO_ERROR: Operation failed due to I/O error (disk read/write failure). OUT_OF_MEMORY: Operation failed as no new memory could be allocated. PROTOCOL: Specific to REST API requests. Request failed due to an HTTP protocol-related error (resource not found, bad request, etc.). The HTTP status provides more specific information about the error. UNSUPPORTED_FEATURE: Requested feature or module not supported by the API, refer to <code>require</code> command. VERSION: Application not supported by the loaded API module, refer to <code>require</code> command. In addition to above, the following error responses indicate incorrect behavior on the part of the API (contact BlueRiver support): - INTERNAL_ERROR - NOT_IMPLEMENTED
message	2.0	String	A error message that is readable by a human, describing the error.

Note: Errors notated above with ** in addition to device-specific error conditions documented for each command individually, also apply to the majority of API commands.

Some commands include an array of device error objects in their return value.

- This allows the API Server to report a command is successfully understood and succeeded for some devices but has not been for other devices.
- Refer to individual commands for details on command specific errors.

A device error object (`DeviceError`) has the following structure:

```
{
  "device_id" : String,
  "reason" : String
  "message" : String
}
```

Table 5: Device API error objects

Member name	Type	Description
device_id	String	Device ID.
reason	String	In addition to the errors outlined in Table 4: API error objects, the following errors can occur in response to a device error occurring. These errors are also machine-readable strings identifying error. DEVICE_DISCONNECTED** Device disconnected from network. DEVICE_TYPE: Operation cannot be performed on device because it does not support it. INVALID_OPERATION: Operation cannot be performed on device for a reason other than device type or state. INVALID_STATE**: Operation cannot be performed on device in its current state. For example a firmware update is in progress. IO ERROR: Operation failed due to I/O error (e.g. disk read/write failure). REQUEST_TIMEOUT**: Device did not respond in time. RESOURCE NOT FOUND: Resource needed for operation not found (e.g. firmware file).
message	String	An error message that is readable by a human, describing the error.

2.6.2 HTTP Status

Each successful API request completes with one of these two HTTP status codes:

1. 200 OK reported when the API request completes immediately and does not lead to the creation of a new resource. In this case, the status member of the response is `SUCCESS`.
2. 201 Created reported in either of these two cases:
 - a. When the API request completes as a background request then the status member of the response is `PROCESSING`. As per the HTTP 1.1 specification, the `Location` header field of the response provides the resource that can be used to monitor the request's progress. This is the HTTP status code provided any time a request has `PROCESSING` status.
 - b. In the event that the request creates a new resource, the status member of the response is `SUCCESS`. The `Location` header field of the response provides the newly created resource. If a specific type of request generates a new resource, then it is explicitly specified in the command reference section.

If the request is not successfully completed, then the status member of the response is `ERROR` and the reason member of the error object in the response maps to the HTTP status code as outlined in the table below.

Table 6: Relation between error reasons and HTTP status codes

Reason	HTTP Status Code
AUTHENTICATION	403 Forbidden
ILLEGAL_ARGUMENT	404 Not Found for invalid/unsupported URI query argument 400 Bad Request otherwise
INTERNAL_ERROR	500 Internal Server Error
INVALID_COMMAND	400 Bad Request
INVALID_STATE	400 Bad Request
IO_ERROR	500 Internal Server Error
MULTIVIEW_LAYOUT_STATE	400 Bad Request
NOT_IMPLEMENTED	500 Internal Server Error
OUT_OF_MEMORY	500 Internal Server Error
PROTOCOL	One of the following: 400 Bad Request 404 Not Found 405 Method Not Allowed 411 Length Required 413 Payload Too Large 415 Unsupported Media Type 505 HTTP Version Not Supported Note: This list is subject to change in the future as features are added to the API.
RESOURCE_NOT_FOUND	400 Bad Request
UNSUPPORTED_FEATURE	404 Not Found
VERSION	404 Not Found

2.6.3 Device Error Objects

Some commands include an array of device error objects in their return value.

- This allows the API Server to report a command is successfully understood and succeeded for some devices but has not been for other devices.
- Refer to individual commands for details on command specific errors.

A device error object (`DeviceError`) has the following structure:

```
{
  "device_id" : String,
  "reason" : String
  "message" : String
}
```

Table 7: Device API error objects

Member name	Type	Description
device_id	String	Device ID.
reason	String	In addition to the errors outlined in Table 4: API error objects, the following errors can occur in response to a device error occurring. These errors are also machine-readable strings identifying error. DEVICE_DISCONNECTED** Device disconnected from network. DEVICE TYPE: Operation cannot be performed because device does not support it. INVALID OPERATION: Operation cannot be performed on device for a reason other than device type or state. INVALID STATE**: Operation cannot be performed on device in its current state. For example a firmware update is in progress. IO_ERROR: Operation failed due to I/O error, such as a disk read/write failure. REQUEST_TIMEOUT**: Device did not respond in time. RESOURCE NOT FOUND: Resource needed for operation not found (e.g. firmware file).
message	String	An error message that is readable by a human, describing the error.

2.7 Description of Streams and Destinations

Control of data routing between devices uses a stream and subscription abstraction:

- Streams are the sending endpoints, while subscriptions are the receiving endpoints.
- Each stream or subscription has an assigned type, only streams and subscriptions of the same type can be associated together.

Depending on their type, streams are either joinable or switchable.

2.7.1 Joinable streams:

Joinable streams are used for the transmission of audio and video data. When an AV stream is started it is configured to send data using a multicast channel assigned by the API Server.

TIP: For further information on network functionality and requirements refer to the SDVoE Solution Technical Overview Guide.

To receive data, the `join` command is used. This command connects/subscribes a decoder (RX) device to the requested multicast channel.

A subscription can be unsubscribed from a multicast channel by either:

1. Joining another subscription.
2. Using the `leave` command.

Joinable streams can be configured to discontinue transmission using the `stop` command.

2.7.2 Switchable streams

Switchable streams are used for RS-232, CEC and Infrared (IR) data.

When configured to directly send data unicast is used to a targeted endpoint device (including the API Server) using the `switch` command. Switch is also used to send data to all devices, but in this case it uses broadcast. The receiving device requires no configuration to receive a switchable stream.

Switchable streams can be configured to discontinue transmission using the `stop` command.

TIP: For information on commands that are applicable to switchable streams refer to section 6 Basic API Commands, specifically the `join`, `leave`, `start`, `stop` and `switch`. For more details on these commands refer to the SDVoE API Developers Reference Guide.

2.8 Multicast IP Address Management

Each active joinable stream uses a multicast IP address. Allocation of multicast IP addresses can be managed either by the client through the API or by the API Server itself. In all cases, the API Server ensures multicast IP address assignments do not conflict.

The client can manage multicast IP address allocation by specifying an address whenever a stream is started with the `start` command.

If no address is specified, the API Server uses either (1) the previously assigned address, if applicable or (2) allocates an unused address.

A list of currently assigned multicast IP addresses can be obtained by calling the `list multicast` command. Whether the multicast IP address associated with a stream was allocated by the client or the API Server, it can be freed by calling the `stop` command with the `free` option.

2.8.1 Multicast Usage

A transmitter can have up to six (6) multicast addresses assigned to it at a given time, one for each of the following:

- Video Stream 0 (Native stream)
- Video Stream 1 (Scaled stream)
- Preview Thumbnail Video Stream (applicable to NT1000/NT2000 chipsets)
- HDMI Audio
- Stereo Audio
- Multichannel Audio

Each receiver can be subscribed with up to four (4) multicast addresses:

- Video
- HDMI Audio
- Stereo Audio
- Multichannel Audio

Note: In addition, important to note that if the receiver is operating in Multiview mode it could be subscribed to more than four multicast channels, multiview supports up to 32 video sources to be composited to a given layout.

For more details on multicast assignments and management. Refer to section 6 Basic API Commands, specifically the `list multicast`, `start` and `stop` commands.

2.8.2 Possible Multicast Address Conflicts

Multicast address conflicts can occur when adding previously configured SDVoE endpoint devices to an existing installation. This section outlines what occurs when transmitter (TX) or receiver (RX) devices are migrated to an existing system and potential concerns that need to be considered. It also outlines recommendations when moving currently configured devices.

2.8.2.1 Migrating an Transmitter (TX)

When an transmitter (TX) with currently assigned multicast addresses, is added to a system where these addresses are presently in use by one or more active transmitter (TX), the API Server automatically assigns new multicast addresses to the decoder that is being added and places the affected streams into a 'stopped' state.

The new transmitter (TX) is however, allowed to 'steal' multicast addresses from another transmitter that has 'disappeared' from the network. If the original device reappears its associated streams are reassigned new unused multicast IP addresses, as well as streams are all stopped.

2.8.2.2 Migrating a Decoder (RX)

When a decoder (RX) is migrated from an existing system to a new system, stream subscriptions currently assigned are not altered.

2.8.2.3 Possible Conflicts

Since a decoder's (RX) subscriptions are unaltered, if a decoder is currently subscribed to a particular feed, it could receive an undesirable stream coming from a newly added transmitter (TX).

For example, if a decoder (RX) is currently configured to receive audio from a given multicast address that is also currently assigned in a newly added transmitter (TX), it could receive video on this address from the new transmitter (TX). If the decoder (RX) is currently already receiving video from another transmitter (TX), this could result in oversubscription.

2.8.2.4 Solution

To prevent undesirable behavior, it is recommended to restore all transmitter (TX) and decoder (RX) devices to factory defaults prior to moving them to another network setup.

If this is not possible then free all multicast addresses on transmitter (TX) and unsubscribe (leave) all subscriptions on decoders (RX) before moving the devices to another setup.

3 Helpful Guidelines

This section reviews some points that are not necessarily obvious but should be kept in mind when creating a control application to interact with the API Server and endpoint devices.

3.1 “get settings” can return PROCESSING status

As an optimization, the API Server caches information that allows it to return a result immediately for the `get settings` command (the `get` command with the subset argument set to `settings`) the majority of the time.

However, applications must not rely on the assumption that this command never returns with a status of `PROCESSING`.

Refer to section 6 Basic API Commands, the `get` command. For more information on the `get` command and supported subsets refer to the SDVoE API Developers Reference Guide. Of particular interest in the Command `get` section is the Return Values, which provides information on which subsets return immediately and which can return with status `PROCESSING`.

3.2 Applying Video Compression to HDMI Native Stream

It is possible to always compress the HDMI native video signal being fed into a transmitter (TX) to reduce the total bandwidth usage in a given network. As outlined in other documentation video signals that surpass the total network bandwidth of 10Gbps are automatically lightly compressed.

Warning! Compressed streams are unavailable for assignment to decoders that are operating in `multiview` mode.

When the `always_compress` argument is enabled (`true`) the HDMI stream 0 is always compressed without regards to the video format. The `video_compressor` node object is used in conjunction with the `set property` command to enable or disable the compression of the native HDMI video stream of a transmitter. Refer to the SDVoE API Developers Reference Guide for more information.

Example, of the command usage to apply to an transmitter (TX):

```
POST /api/device/{device_id}
{
  "key" :
  "nodes[VIDEO_COMPRESSION:0].configuration.always_compress",
  "value" : Boolean
}
```

Example of using the `true` value with the video compression node object to enable video compression:

```
POST /api/device/d88039e55d59 HTTP/1.1
...
{
  "key" :
  "nodes[VIDEO_COMPRESSION:0].configuration.always_compress",
  "value" : true
}
```

Example of using the `false` value with video compression node object to disable video compression:

```
POST /api/device/d88039e55d59 HTTP/1.1
...
{
  "key" :
  "nodes[VIDEO_COMPRESSION:0].configuration.always_compress",
  "value" : false
}
```

Reminder: The compression option is only available for the native stream (stream 0). No compression is supported for the scaled stream (stream 1).

3.3 Removing HDMI Audio from HDMI Video Stream

The native HDMI video stream (stream 0) can and often does contain not only video but also additional data, such as color format, timestamp and other overhead data as well as auxiliary audio data.

It is possible to disable the transfer of the HDMI auxiliary audio data embedded within the HDMI video stream through the default hardware configuration settings. The default settings are applied during a firmware update, factory reset or initial/first time programming. The HDMI auxiliary audio data setting can be modified through the API using the set property command with the key string, `streams[stream_type:index].configuration.remove_audio`.

When the `remove_audio` argument is enabled the HDMI audio is always removed from the HDMI stream 0, so it is necessary to subscribe a receiver (RX) to the appropriate audio stream using the `join` command. This applies even when a receiver is operating in Genlock mode. Refer to the SDVoE API Developers Reference Guide for more information on this property node.

TIP: Refer to section 6 Basic API Commands, specifically the `join` command for more information on subscribing/joining streams.

Example, of the command usage to apply to an transmitter (TX):

```
POST /api/device/{device_id}
{
  "key" : "
  "streams[stream_type:index].configuration.remove_audio",
  "value" : Integer
}
```

Example of enabling the remove audio stream option:

```
POST /api/device/d88039e55d59 HTTP/1.1
...
{
  "key" : "
  streams[stream_type:index].configuration.remove_audio",
  "value" : 1
}
```

Example of using the `false` value with video compression node object to disable video compression:

```
POST /api/device/d88039e55d59 HTTP/1.1
...
{
  "key" : "
streams[stream_type:index].configuration.remove_audio",
  "value" : 0
}
```

Reminder: The remove audio option is only available for the native stream (stream 0), not applicable to the scaled stream (stream 1).

3.4 Guidelines for use of the SDVoE API

This section provides guidelines on the use of the SDVoE REST API.

3.4.1 Checking for Supported Features

As new versions of the API become available, new features are added. Therefore, it is recommended that the first step performed by a client is to confirm that the specific instance of the API Server with which it is to interact supports the expected feature set.

Refer to section 2.2 API Versioning and Application Compatibility and section 6 Basic API Commands for information on the `require` command.

If the requested feature set is not supported, the request fails with an error of either:

UNSUPPORTED_FEATURE	If one or more of the named modules are not supported.
VERSION	If the specified version is not supported.

Example of a REST API request:

```
GET /api?require(api)=3.2.0.0
{
  "status": "SUCCESS",
  "request_id": null,
  "result": {
    "server": "Control Server",
    "vendor": "Semtech",
    "version": "3.2.0.0",
    "modules": [
      {
        "name": "api",
        "version": "3.2.0.0"
      }
    ]
  },
  "error": null
}
```

3.4.2 Monitoring Event Notifications

If asynchronous notifications are to be handled in the context of an in-browser web application, the use of a WebSocket connection is encouraged. When a WebSocket connection is used for monitoring asynchronous event notifications, once the connection is established the API begins sending notification messages. It is not possible to receive asynchronous (i.e. server push) event notifications without the use of a WebSocket due to the request/response nature of the HTTP protocol.

Reminder monitoring asynchronous notifications generated by the API notifies clients of changes and events that have occurred that impact endpoint devices. For examples of notifications that occur when a device is discovered by the API, refer to section 7 Device Discovery and Feature Detection.

Alternately, through the REST API itself, events can only be handled by polling with the `event` command. If polling is being utilized the `event` command is invoked to retrieve a list of current events. Refer to section 2.3.4 Polling Events for more information.

Reminder: The API Server buffers command returns and notification events, refer to section 2.3.3 Event Buffering for details.

3.4.3 Establishing Communication between a Client and the API

When planning the design of client application using WebSocket connections, it is necessary for clients to establish a connection with the API Server. This section provides an understanding of the process that is typically used to establish communication between a client application and the API Server.

3.4.3.1 Establishing Connection from a Client

For the WebSocket connection, the resource to connect to is `/api`. As an example, if the host name of the machine hosting the API Server is “example.com”, connect to URI `wss://example.com/api` (with TLS) or to URI `ws://example.com/api` (without TLS).

3.4.3.2 Enabling Human-Readable Output (mode human)

This step is optional. API responses can either be indented for easy readability or minified, that is contain only the required characters with all whitespace removed. The default behavior is dependent on the configuration of the API server.

The server default can also be overwritten in any REST API request on a per-request basis by providing the `human` query argument. This is useful when performing GET requests manually in a web browser, such as during development.

A user can invoke API requests manually, which is often useful during development or troubleshooting. Invoking the `human mode` command indents the API responses for better readability. Should be noted however that this command is optional and not normally necessary for clients to invoke it.

To enable the human mode, use the command `...?human=true`

The command returns with a `SUCCESS` response similar to the example shown below.

```
GET /api?human=true HTTP/1.1
...
HTTP/1.1 200 OK
...
```

```

{
  "status": "SUCCESS",
  "request_id": null,
  "result": {
    "server": "Control Server",
    "vendor": "Semtech",
    "version": "3.2.0.0",
    "modules": [
      {
        "name": " api",
        "version": "3.2.0.0"
      }
    ]
  },
  "error": null
}

```

3.4.3.3 Load API Command Module

Once a connection is established from the client application to the API, next it is necessary to load the appropriate API command module. This step ensures the version of the API Server running is compatible with the client application.

This module is loaded using the `require` command.

Command syntax:

```

GET /api?require(module_name)=version
module_name = requested module
version = version number

```

The SDVoE API and BlueRiver chipset firmware are typically released together. In the release notes, the latest versions of all components are indicated. Important to note that unless otherwise stated in the release notes the latest API supports previous versions of firmware, but the opposite is not necessarily true – that is that the latest firmware is not necessarily supported by earlier versions of the API.

The best practice is to always update both, chipset firmware and API versions, to ensure the latest functionality and fixes are available. Refer section 2.2 API Versioning and Application Compatibility for more information on compatibility.

The example syntax shown below loads the API version 3.1.1.0, declaring that the client application's command set is based on this version.

```

GET /api?require(api)=version 3.1.1.0
{
  "status": "SUCCESS",
  "request_id": null,
  "result": {
    "server": "Control Server",
    "vendor": "Semtech",
    "version": "3.2.0.0",
    "modules": [
      {

```

```
        "name": "api",
        "version": "3.2.0.0"
    }
  ],
  "error": null
}
```

Refer to the SDVoE API Developers Reference Guide for more information on the `require` request.

3.5 Enabling and Disabling the Frame Rate Converter

When managing the 10GbE bandwidth, for either the input of a decoder (RX) or the output of the transmitter (TX), it could be necessary to reduce bandwidth usage. Enabling frame rate conversion on the transmitter (TX) is useful in reducing the total amount of bandwidth being used for video transmission. Applying frame rate conversion allows for a source's frame rate to be divided by two on the native stream (stream 0) and/or scaled stream (stream 1).

Frame rate conversion is managed using the `set property` command and can be invoked either when a connection is established or while a session is active.

For details and examples on using the `set property` command refer to the SDVoE API Application Note Library and the SDVoE API Developers Reference Guide in the `set property` command section.

4 Core Commands

SDVoE API supports an extensive set of commands designed to support the functionality built into its Pro-AV hardware chipsets. The table provided below reviews the commands that can be issued “prior” to loading an API command module.

These commands modify the behavior of the API. On the WebSocket connection their effects are limited to a single API connection. For a full list of the supported base API commands refer to Table 13: List of API commands.

Table 8: API Command Modes

Command	Return Value	Description and Errors
mode human	null	Enable or disable human-readable output mode. When human mode enabled, JSON responses returned by API are indented and formatted to be more easily read. When disabled, JSON output contains no newline, space or other white space characters. Example provided in section 3.4.3.2 Enabling Human-Readable Output (mode human). ILLEGAL_ARGUMENT occurs when a value other than on or off is specified.
quit	null	Requests that the API server close the connection. Applicable to TCP and web socket connections. Optionally, client can simply close TCP connection instead. INVALID_COMMAND indicates the API Server supports only a version of the API prior to 2.5.
require	null	Verifies requested version of command module available in the current API Server and then loads specified module if it is. Notes: i. Compatibility rules outlined in section 2.2 API Versioning and Application Compatibility. ii. Section 3.4.3.3 Load API Command Module provides examples of loading these modules. Errors: VERSION returned when API version not compatible with version requested. UNSUPPORTED_FEATURE returned when requested module is not supported by the API version.
version	Four possible return values: server vendor version modules	Get version and other data about the API Server and available command modules. INVALID_COMMAND indicates that the API Server supports only an API version prior to 2.4. Note: Refer to Table 9 below for details.

Table 9: Version command return values

Member name	Since	Type	Description
server	2.4	String	Name of API server software.
vendor	2.4	String	Vendor of API server software.
version	2.4	String	Release version of API server software.
modules	2.4	Array of ModuleDescriptionObject	Supported API command modules. Refer to above table.

The `version` command has a `ModuleDescriptionObject` JSON object with the following structure:

```
{
  "name" : String,
  "version" : String
}
```

Table 10: Version JSON Module description Object

Member name	Since	Type	Description
name	2.4	String	Name of API command module.
version	2.4	String	Version of API command module.

5 Managing the Video Output Format

When planning a control application that is interact with SDVoE endpoint devices, a thorough understanding in relation to managing the video output format with the API is required. This section provides an introduction to managing video output arguments, as well as outlines the supported VIC/HDMI_VIC codes.

5.1 Specifying a Video Output Format

Managing the video output format is a major factor of the API command module. There are three command options available to manage the video output format of a decoder (RX) device.

These command options are:

1. `set video` command.

This is generally the recommended command to use to specify the video output format arguments including the display mode, video format and frame rate.

2. `join` command.

When joining a video stream, the video output format can optionally be specified. This allows both operations (joining and setting video output format) to be performed with a single command.

3. `set multiview` command.

Used to set the device in multiview mode with a specified multiview layout.

Section 6 Basic API Commands provides an overview on each of these commands.

For more detailed information on the use of these commands refer to the SDVoE API Developers Reference Guide.

5.1.1 Video Display Modes

Both the `set video` and `join` commands (when a video format is specified, which is optional for the `join` command) require a video display mode argument.

The display mode argument is a string that includes one of the following values:

GENLOCKED	Specifies video mode is to be set to genlock (i.e. synchronized zero-frame latency) full screen 1:1 display mode.
GENLOCK_SCALING	Specifies the video mode is to be set to genlock (i.e. zero-frame latency) full screen mode with scaling.
FAST_SWITCHED	Specifies the video mode is to be set to fast switched full screen mode, with aspect ratio preserved by adding black bars when needed.
FAST_SWITCHED_CROP	Specifies the video mode is to be set to fast switched full screen mode, with aspect ratio preserved by cropping the video.

FAST_SWITCHED_STRETCH	Specifies the video mode is to be set to fast switched full screen mode, with the video stretched to cover the entire screen without regard for preserving the aspect ratio.
WALL_GENLOCKED	Specifies the video mode is to be set to genlocked (synchronized) display wall mode.
WALL_FAST_SWITCHED	Specified the video mode is to be set to fast switched display wall mode.

Note that multiview mode is not listed as an option. The `set multiview` command is used to configure a device that is operating in multiview mode. Refer to the `set multiview` command in section 6 Basic API Commands for more information.

5.1.2 Video Format Arguments

In an API request, the video arguments are specified as a `VideoFormatArguments` JSON object. The format of this object type is as follows:

```
{
  "aspect_ratio" : String (optional),
  "fps" : Real or String (see text, conditional),
  "hsync_front_porch" : Integer (conditional),
  "vsync_front_porch" : Integer (conditional),
  "hdmi_vic" : Integer (conditional),
  "keep_width" : Integer (conditional),
  "keep_height" : Integer (conditional),
  "horiz_offset" : Integer (conditional),
  "vert_offset" : Integer (conditional),
  "horiz_polarity" : String (conditional),
  "vert_polarity" : String (conditional),
  "hsync_pulse" : Integer (conditional),
  "vsync_pulse" : Integer (conditional),
  "width" : Integer (conditional),
  "height" : Integer (conditional),
  "total_width" : Integer (conditional),
  "total_height" : Integer (conditional),
  "vic" : Integer (conditional),
  "viewport_horiz" : Integer (conditional),
  "viewport_vert" : Integer (conditional),
  "viewport_width" : Integer (conditional),
  "viewport_height" : Integer (conditional),
  "quantization_range" : String (optional)
  "aspect_ratio_mode" : String (conditional),
  "grid_width" : Integer (conditional),
  "grid_height" : Integer (conditional),
  "grid_index_horiz" : Integer (conditional),
  "grid_index_vert" : Integer (conditional),
  "bezel_horiz" : Integer (conditional),
  "bezel_vert" : Integer (conditional)
  "wall_name": : String (optional)
}
```

Table 11: API Video Format Arguments below outlines the API video format arguments supported along with a description. For more detailed information on the use of these format arguments refer to the SDVoE API Developers Reference Guide.

Unless otherwise stated, arguments apply to all display modes except Genlock, which does not require any video format arguments to be specified. Also unless otherwise stated, all arguments are individually optional, but a minimum set of arguments must be specified to uniquely identify the video format. For more information refer to section 5.1.3 Uniquely Identifying a Video Format.

Also, note that if the Source Independent Wall API arguments are being applied, the group of arguments must be specified and that this option is only applicable to the AVP2000(T) decoder (RX) devices. For more information on this option refer to the API Developers Application Note Library.

The following subsections describe the possible video format arguments that can be specified.

Table 11: API Video Format Arguments

Format Argument	Description
Picture Aspect Ratio	<code>aspect</code> argument specifies the picture aspect ratio. Two valid formats for this argument, either: 1. Directly specify the aspect ratio as a ratio, such as 4:3 or 16:9. 2. Specify one of the following: <code>ASPECT_NO_DATA</code> If aspect ratio is to be left unspecified. <code>ASPECT_4_3</code> To specify a 4:3 aspect ratio. <code>ASPECT_16_9</code> To specify a 16:9 aspect ratio. <code>ASPECT_256_135</code> To specify a 256:135 aspect ratio. <code>ASPECT_64_27</code> To specify a 64:27 aspect ratio.
Frame Rate	<code>fps</code> argument specifies the frame rate to apply in frames per second (fps). Argument can be either an integer (e.g. 50) or real number (e.g. 50.000) that represents the of frames per second or optionally a string with the letter “m” (e.g. 60m or 30.00m) to indicate frame rate must be divided by 1.001 (i.e. multiplied by 1000/1001). Reminder: When a receiver is operating in genlock, genlock scaling or genlock wall mode, frame rate must match frame rate of video source. Device must also support specified video mode.
Video Sync Front Porch	<code>front_porch</code> argument sets the <code>hsync_front_porch</code> and <code>vsync_front_porch</code> respectively, specifying horizontal and vertical front porch, in pixels. If either front porch argument is specified then both must be. Note: If front porch specified, <code>pulse_width</code> must also be specified.
HDMI Video Identification Code	<code>hdmi_vic</code> argument specifies HDMI Video Identification Code (HDMI_VIC). The supported values are: 1 -- 3840x2160 (2160p) 29.97Hz or 30Hz 2 -- 3840x2160 (2160p) 25Hz

Format Argument	Description
	3 -- 3840x2160 (2160p) 24Hz or 23.98Hz 4 -- 4096x2160 24Hz or 23.98Hz (256:135) Other values are inserted in the HDMI Vendor-Specific InfoFrame as requested but cannot be used to uniquely identify a video format.
Keep Region Size	keep argument sets keep_width and keep_height respectively, specifying width and height of the portion of source video that is to be displayed (i.e. "kept"), in pixels. If either keep region size argument is specified then both must be. The keep_width argument must be even, that is a multiple of two. keep argument specific to genlock wall and fast switch wall display modes and typically is specified with the read offset argument. It is not specified for any other display mode.
Read Offset	offset argument sets horiz_offset and vert_offset respectively, specifying horizontal and vertical offset of displayed video from the full video source, in pixels. If either offset argument is specified then both must be. offset argument is specific to genlock wall and fast switch wall modes and typically is specified along with the keep region size argument. It is not specified for other display modes.
Video Sync Polarity	polarity argument sets horiz_polarity and vert_polarity respectively, specifying polarity of horizontal and vertical synchronization pulse. Each may be set to either: POSITIVE -- positive sync polarity NEGATIVE -- negative sync polarity If either polarity argument is specified then both must be.
Video Sync Pulse Width	pulse argument sets hsync_pulse and vsync_pulse respectively, specifying width of horizontal and vertical synchronization pulse. If either video sync pulse width argument is specified then both must be. Note: If pulse width is specified, front porch must also be specified.
Video Resolution	size argument specifies width and height respectively, specifying width and height of the displayed video, in pixels. If either size argument is specified then both must be. The width argument must be even, that is a multiple of two. Note: Optional for multiview display mode. If not specified, video resolution set to match the multiview layout size.
Total Size	total argument specifies total width and total height arguments respectively, specifying total width and height in pixels including the blanking. If either the total width or height argument is specified then both must be.

Format Argument	Description
Video Identification Code (VIC)	<code>vic</code> argument specifies the Video Identification Code (VIC). Values <i>not</i> specified in Table 12 are inserted in Auxiliary Video Information (AVI) InfoFrame as requested but cannot be used to uniquely identify the video format. Table 12: Supported Video Identification Codes (VIC) below provides the list of supported VIC codes.
Viewport	<code>viewport</code> is a rectangular region on-screen where video is rendered, areas outside this region are black. This argument specific to genlock wall and fast switch wall video modes and is not applicable to other video modes. Refer to the SDVoE API Developers Reference Guide and the SDVoE API Application Note Library for more information on the viewport argument.
Color Quantization Range	<code>quantization</code> specifies color quantization range of the output. The following options are supported: <code>STANDARD</code> Appropriate quantization range for the video format as defined by the CEA-861-F standard. <code>AUTO</code> Same as <code>STANDARD</code> but explicitly state <code>FULL</code> or <code>LIMITED</code> as appropriate instead of <code>DEFAULT</code> in the AVI InfoFrame. This can be used to work around some monitor issues. <code>LIMITED</code> Force limited quantization range. <code>FULL</code> Force full quantization range. If this argument is omitted, the behavior is the same as if <code>STANDARD</code> was specified.
SOURCE INDEPENDENT WALL (auto wall config)	The arguments outlined below apply to a <code>set video</code> request that is to configure a Video Wall using the Source Independent Wall option. When a video wall is being built using these arguments, reminder that each argument listed below must be specified. Also, this option is applicable to AVP2000(T) decoder devices only. Refer to the API Developers Application Note Library for more information on this feature.
<code>aspect_ratio_mode</code>	Specify the aspect ratio to be applied to the output stream. Options are <code>KEEP</code> , <code>CROP</code> and <code>STRETCH</code> .
<code>grid_width</code>	Specify the total number of columns that are included in the wall grid.
<code>grid_height</code>	Specify the total number of rows that are included in the wall grid.
<code>grid_index_horiz</code>	The horizontal index of the display in the wall grid. Example, if the wall grid is 3x2 and this is the second display mounted horizontally the index would be 2.
<code>grid_index_vert</code>	The vertical index of the display in the wall grid. Example, if the wall grid is 3x2 and this is the first display mounted vertically, the index would be 1 and mounted in row 2 would be index 2.
<code>bezel_horiz</code>	The value of the horizontal bezel compensation to be applied, in pixels.
<code>bezel_vert</code>	The value of the vertical bezel compensation to be applied, in pixels.

Format Argument	Description
wall_name	The wall name can be used by control applications to verify which endpoint devices are part of a given video wall configuration. This is useful when multiple video walls exist within the same SDVoE system.

The figure shown below is an illustration of the grid indexes for a Video Wall of 3x2 displays.

1,1	2,1	3,1
1,2		
1,3		

Figure 5: Illustration of Grid Indexes for a 3x2 Video Wall

Table 12: Supported Video Identification Codes (VIC)

Video Identification Code	Video Format
1	640x480 60Hz (4:3)
2	720x480 59.94Hz or 60 Hz (4:3)
3	720x480 59.94Hz or 60 Hz (16:9)
4	1280x720 (720p) 59.94Hz or 60 Hz
16	1920x1080 (1080p) 59.94Hz or 60 Hz
17	720x576 50Hz (4:3)
18	720x576 50Hz (16:9)
19	1280x720 (720p) 50Hz
31	1920x1080 (1080p) 50Hz
32	1920x1080 (1080p) 24Hz or 23.98Hz
33	1920x1080 (1080p) 25Hz
34	1920x1080 (1080p) 29.97Hz or 30Hz
60	1280x720 (720p) 24Hz or 23.98Hz
61	1280x720 (720p) 25Hz
62	1280x720 (720p) 29.97Hz or 30Hz
93*	3840x2160 (2160p) 24Hz or 23.98Hz
94*	3840x2160 (2160p) 25Hz
95*	3840x2160 (2160p) 29.97Hz or 30Hz
96	3840x2160 (2160p) 50Hz
97	3840x2160 (2160p) 59.94Hz or 60 Hz
98*	4096x2160 24Hz or 23.98Hz (256:135)

Video Identification Code	Video Format
99	4096x2160 25Hz (256:135)
100	4096x2160 29.97Hz or 30Hz (256:135)
101	4096x2160 50Hz (256:135)
102	4096x2160 59.94Hz or 60 Hz (256:135)

- * Video Identification Codes (VIC) with an asterisk can alternatively be specified by their HDMI Video Identification Codes (HDMI_VIC). Refer to the HDMI Video Identification Code description in Table 11: API Video Format Arguments.

5.1.3 Uniquely Identifying a Video Format

To uniquely identify a video format a minimum set of video format arguments are required.

This following sub-sections describe the various options available to identify a video format.

5.1.3.1 By Video Identification Code (VIC or HDMI_VIC)

If the desired video format has a supported Video Identification Code (VIC) or HDMI Video Identification Code (HDMI_VIC), it is sufficient to specify this information. Other specific parameters, such as video timings, are automatically retrieved from a table.

For some video formats, there is an ambiguity regarding the frame rate. This is resolved as follows:

- Formats with a frame rate of 24Hz or 23.98Hz (24Hz/1.001), the 24Hz frame rate is selected.
- Formats with a frame rate of 30Hz or 29.97Hz (30Hz/1.001), the 29.97Hz frame rate is selected.
- Formats with a frame rate of 60Hz or 59.94Hz (60Hz/1.001), the 59.94Hz frame rate is selected.

Refer to Table 12: Supported Video Identification Codes (VIC) supplied above for information on the Video Identification Code (VIC) supported codes and to Table 11: API Video Format Arguments for HDMI Video code (HDMI_VIC) supported codes.

5.1.3.2 By Video Identification Code (VIC or HDMI_VIC) and Frame Rate

To uniquely identify the video format using the Video Identification Code (VIC) or HDMI Video Identification Code (HDMI_VIC) along with a frame rate, is the same as outlined for “by Video Identification Code (VIC or HDMI_VIC)” but the frame rate ambiguity is resolved by explicitly specifying the frame rate.

5.1.3.3 By Video Resolution and Frame Rate

If the desired video format has a supported Video Identification Code (VIC) or HDMI Video Identification Code (HDMI_VIC), it is sufficient to specify only the video resolution and frame rate.

5.1.3.4 Completely Specified Video Formats

If the video format does *not* have a supported Video Identification Code (VIC) or HDMI Video Identification Code (HDMI_VIC), all video arguments must be specified to completely describe the video format.

This specifically includes the following:

- Picture aspect ratio if applicable
- Frame Rate
- Video Sync Front Porch
- Video Sync Polarity

-
- Video Sync Pulse Width
 - Video Resolution
 - Total Size

Refer to Table 11: API Video Format Arguments for a description of each of these arguments.

Semtech Confidential Information
Mark Li
ADV
02/03/2026

6 Basic API Commands

The `api` command module contains a series of commands that are used for controlling and querying the status of SDVoE transmitter, receiver and/or transceiver devices.

Table 13 provides a list of some of the commonly used commands supported by the API, along with a brief description of each command's role.

TIP: This list is not exhaustive. For detailed information on API commands, including syntax, arguments and examples refer to the SDVoE API Developers Reference Guide.

Reminder: Before the commands described in this section can be called, the API command module must first have been loaded with the `require` command..

Table 13: List of API commands

Command	Description								
<code>auth</code>	Security feature that allows for secure communication between an API server and endpoint devices.								
<code>claim</code>	Associates target device(s) with a API Server from which command is issued so it is controlled exclusively through this API instance. An unauthorized API server on the network will not be able to control a claimed device. The effect of this command is undone with the <code>release</code> command.								
<code>event</code>	Retrieves list of events that occurred recently. Arguments available to filter since last query issued, as well as to retrieve specified number of events. The <code>type</code> query argument provides the ability to filter events based on their event type. The <code>settings_changed</code> query argument provides the ability to filter events based on the setting change subset(s) that actually changed. Refer to the SDVoE API Developers Reference Guide for more information.								
<code>get</code>	The <code>get</code> command fetches information from a single or group of devices. For each device, information retrieved is a subset of the device object. This object contains all configuration, status, supported features and other information for each device.								
<code>join</code>	Subscribe one or more device(s) to a stream. Stream types supported: <table><tr><td>HDMI</td><td>HDMI video stream</td></tr><tr><td>HDMI_AUDIO</td><td>HDMI audio stream</td></tr><tr><td>MULTICH_AUDIO</td><td>Multichannel audio stream</td></tr><tr><td>STEREO_AUDIO</td><td>Stereo audio stream</td></tr></table>	HDMI	HDMI video stream	HDMI_AUDIO	HDMI audio stream	MULTICH_AUDIO	Multichannel audio stream	STEREO_AUDIO	Stereo audio stream
HDMI	HDMI video stream								
HDMI_AUDIO	HDMI audio stream								
MULTICH_AUDIO	Multichannel audio stream								
STEREO_AUDIO	Stereo audio stream								
<code>leave</code>	Unsubscribe one or more device(s) from a stream.								
<code>list</code>	Two options available for the <code>list</code> command: <code>list firmware</code> List all firmware update files available to the API server. <code>list multicast</code> List information regarding the multicast IP addresses currently in use.								

Command	Description
netstat	<code>netstat read</code> retrieves network characteristics for one or more endpoint devices. Data is useful in investigating possible cause of a network problem. <code>netstat clear</code> resets all counters. Refer to the SDVoE API Developers Reference Guide for details on the <code>netstat</code> command.
pair	Pair USB 2.0 local extender (LEX) to remote extender (REX).
reboot	Causes the target device(s) to restart. Note: The target device(s) are unavailable for a short period of time while the restart is completed.
release	Undo the effect of the <code>claim</code> command. This can only be done from the API Server on which the device was claimed.
request	This function retrieves either the return value of a <code>background</code> command or the data associated with an event. <code>REQUEST_COMPLETE</code> event indicates completion of a background command (i.e. command responded with status <code>PROCESSING</code> and a request ID instead of status <code>SUCCESS</code>). Reminder: Background commands respond with <code>PROCESSING</code> status and a request ID, for details refer to section 2.4.1 Background Commands. Format of return same as output of the “<code>get settings</code>” command that would have been called on the device associated with the <code>SETTINGS_CHANGED</code> event.
send	The <code>send</code> command provides three sub-commands which can be used to send the following data types to one or more device(s). 1. Send Consumer Electronics Control (CEC) data to one or more device(s). 2. Send infrared remote-control data to one or more device(s). 3. Send RS-232 data to one or more device(s).
send_wol	The <code>send_wol</code> command is used to send a Wake-on-Lan (WoL) packet from the API to an AVP device, for the AVP device to relay it on the network (acts as a WoL proxy) by broadcasting the message to the local subnet to wake a powered-off target machine. This feature was added in API revision 3.7 and ASIC firmware version 2.2. Applicable to BlueRiver AVP devices.
set	The <code>set</code> command is a powerful command that has a variety of options to manage transmitters and receivers. Below provides a description some of these options. <code>set edid</code> Set the EDID of one or more transmitter device(s). <code>set ip</code> Set the Internet Protocol (IP) configuration of the primary network interface of a target device. <code>set property</code> Set a configuration property for one or more device(s). For more information on configuration properties, refer to the SDVoE API Developers Reference Guide. <code>set scaler</code> Set the scaler video resolution of one or more transmitter device(s). <code>set video</code> Set the video mode and video output parameters of one or

Command	Description
	more receiver device(s).
start	Used to start one or more stream(s) on one or more device(s). HDMI HDMI video stream HDMI_AUDIO HDMI audio stream MULTICH_AUDIO Multichannel audio stream STEREO_AUDIO Stereo audio stream
stop	Stops one or more stream(s) on one or more device(s) and optionally free the multicast address associated with each stopped stream so it can be re-used for another stream.
switch	Set stream destination for a device. switch command used to establish a one-way connection between source and destination devices. To establish two-way connection between two devices, the command must be issued twice with source and destination reversed. There is three options that can be applied to specify the destination. These options are: 1. Using device ID and stream index when destination is a device. 2. Using the keyword "api" when destination is API Server. 3. Specifying the destination IP address.
unauth	Returns the device(s) to a non-secured, unauthenticated operational status.
update	Update the BlueRiver chipset firmware of one or more devices. While a firmware update is in progress, no other operation can be performed on the device. For this reason, any command sent to the API Server that targets the device will fail with INVALID_STATE for this device until the firmware update completes. All connected clients are notified of the start and end of a firmware update with a FIRMWARE_UPDATE_START event and a FIRMWARE_UPDATE_COMPLETE event, respectively. A reboot command must be issued to the device once the firmware update is complete for the changes to take effect. The device is in an undefined state until that reboot.
usb	The set command has several sub-commands to manage the USB HID extension on a device. Below outlines the sub-commands providing a brief outline of each. usb descriptor read Read USB descriptors usb descriptor write Write USB descriptors

This page is intentionally blank.

Semtech Confidential Information
Mark Li
ADV
02/03/2026

7 Device Discovery and Feature Detection

This section provides an example of asynchronous event notifications being received for two devices that are active on the network (DEVICE_CONNECTED events).

Reminder: An event notification displays information about the devices that the API Server has currently detected (DEVICE_INFO_AVAILABLE events). For information on asynchronous event notifications refer to section 2.3.5 Handling Asynchronous Events.

```
{
  "status" : "NOTIFICATION",
  "request_id" : null,
  "result" : {
    "events" : [
      {
        device_id: 0016c04c4cc1,
        event_id: 9,
        event_type: DEVICE_CONNECTED,
        timestamp: 1595606387,
        request_id: 9
      },
      {
        device_id: 0016c04c4cc1,
        event_id: 10,
        event_type: DEVICE_INFO_AVAILABLE,
        timestamp: 1595606387,
        request_id: 10
      },
      {
        device_id: "001aae05cf5f",
        event_id: 11,
        event_type: "DEVICE_INFO_AVAILABLE",
        timestamp: 1595606387,
        request_id: 11
      },
      {
        device_id: "001aae05cf5f",
        event_id: 12,
        event_type: "DEVICE_INFO_AVAILABLE",
        timestamp: 1595606387,
        request_id: 12
      }
    ]
  },
  "error" : null
}
```

When a notification is received, a `request` command can then be sent to retrieve the details of the event. In this example, for device 1 (0016c04c4cc1) the `request_id` is 10 and the event type is `DEVICE_INFO_AVAILABLE`. In response to the `request` command, the API retrieves the detailed data contained in the device info event for device 1.

TIP: The same information can also be retrieved later using the `get device` command.

Looking closer at the provided example, the following data is of particular interest for device 1 (0016c04c4cc1/ request ID 10):

Under the `devices: identity` section, note that the `chipset_type` shown for this example is an AVP2000T.

Also, referring to this section note the device type is confirmed as a `transceiver` device, as `true` is shown for both the receiver and transmitter options.

```
is_receiver: true,  
is_transmitter: true,
```

In addition, under `capabilities`: the return shows the presence of an HDMI Input and HDMI output:

```
hdmi_inputs: 1,  
hdmi_outputs: 1,
```

The `streams` : section is often a useful category in a JSON return. In the example shown here, under the `streams` : section, note that there is two HDMI streams indicated as being present, index 0 and 1.

Also, note that the HDMI index 0 stream (native resolution stream) shows a state of `streaming`, while the second index 1 (scaled resolution stream) shows a state of `stopped`.

It is possible to confirm what hardware interfaces are enabled in the hardware configuration reported from the device to the API as well.

Looking at our example return for the `type`: RS232 for instance, we can confirm that there are four (4) RS-232 ports indicated. This is because when viewing the RS-232 type there is four entries present, index 0 through 3.

Example shown of index: 0.

```
type: RS232,  
index: 0,
```

TIP: Items may be listed in more than one section of a JSON return. For example, when present RS232 (UART) ports are listed not only in the `type` category but also under `capabilities`. A entry is present `uarts: 4`, indicating there is four RS232 ports present.

Following is a JSON return of a `DEVICE_INFORMATION_AVAILABLE` event for the device ID 0016c04c4cc1 used in the above examples .

```
{  
  status: SUCCESS,  
  request_id: 518,  
  result: {  
    devices: [  
      {  
        device_id: 0016c04c4cc1,  
        device_name: my_device_c1,  
        identity: {  
          chipset_type: AVP2000T,  
          engine: PLETHORA,  
          vendor_id: 1,  
          product_id: 88,  
          firmware_comment: BlueRiver_RDK_2-4-0-0_evaluation,  
          firmware_version: 2.4.0.0,  
        }  
      }  
    ]  
  }  
}
```

```

        firmware_rc: 0,
        is_receiver: true,
        is_transmitter: true,
        firmware_details: [
            {
                type: PRIMARY,
                details_available: true,
                firmware_version: 2.4.0.0,
                firmware_rc: 0
            },
            {
                type: GOLDEN,
                details_available: true,
                firmware_version: 2.1.0.0,
                firmware_rc: 0
            }
        ]
    },
    capabilities: {
        color_generator_force: false,
        dhcp_option_43: true,
        dot1x: true,
        external_video_compression_supported: false,
        frame_rate_divider: false,
        freeze_video: true,
        hdmi_audio_downmix: true,
        hdmi_inputs: 1,
        hdmi_outputs: 1,
        hdmi_stream1_auto_stop: true,
        infrared_inputs: 1,
        infrared_outputs: 1,
        local_loopback: true,
        multich_audio_inputs: 1,
        multich_audio_outputs: 1,
        nbaset: false,
        net10gbe_dual_link: false,
        overlay: false,
        scaler: true,
        send_wol: true,
        stereo_audio_inputs: 0,
        stereo_audio_input_outputs: 1,
        stereo_audio_outputs: 0,
        thumbnail: false,
        uarts: 4,
        usb_hid: true,
        usb_extender: true,
        video_processing: true
    },
    configuration: {
        device_name: my_device_c1,
        locate_mode: false,
        secure_av: default,
        resume_streaming: true,
        claimed: false,
        authenticated: false,
        device_mode: TRANSCEIVER
    }
}

```

```

    },
    status: {
        active: true,
        point_to_point: false,
        point_to_point_peer: ,
        boot_status: PRIMARY,
        update_in_progress: false,
        igmp_reporting_mode: INTERVAL,
        error_status: {
            has_error_code: false,
            error_code: 0
        },
        temperature: 58
    },
    streams: [
        {
            type: HDMI,
            index: 0,
            configuration: {
                address: 224.1.1.22,
                enable: true,
                auto_stop: false,
                remove_audio: false
            },
            status: {
                state: STREAMING
            },
            source: {
                ref_class: NODE,
                ref_type: HDMI_DECODER,
                ref_index: 0
            }
        },
        {
            type: HDMI,
            index: 1,
            configuration: {
                address: 0.0.0.0,
                enable: false,
                auto_stop: false,
                remove_audio: true
            },
            status: {
                state: STOPPED
            },
            source: {
                ref_class: NODE,
                ref_type: SCALER,
                ref_index: 0
            }
        },
        {
            type: HDMI_AUDIO,
            index: 0,
            configuration: {
                address: 0.0.0.0,

```

```

        enable: false,
        auto_stop: false
    },
    status: {
        state: STOPPED
    },
    source: {
        ref_class: NODE,
        ref_type: HDMI_DECODER,
        ref_index: 0
    }
},
{
    type: MULTICH_AUDIO,
    index: 0,
    configuration: {
        address: 0.0.0.0,
        enable: false,
        auto_stop: false
    },
    status: {
        state: STOPPED
    },
    source: {
        ref_class: NODE,
        ref_type: MULTICH_AUDIO_INPUT,
        ref_index: 0
    }
},
{
    type: STEREO_AUDIO,
    index: 0,
    configuration: {
        address: 0.0.0.0,
        enable: false,
        auto_stop: false
    },
    status: {
        state: STOPPED
    },
    source: {
        ref_class: NODE,
        ref_type: STEREO_AUDIO_INPUT,
        ref_index: 0
    }
},
{
    type: RS232,
    index: 0,
    configuration: {
        address: 0.0.0.0,
        dest_index: 0,
        enable: false,
        auto_stop: false
    },
    status: {

```

```

        state: STOPPED
    },
    source: {
        ref_class: NODE,
        ref_type: UART,
        ref_index: 0
    }
},
{
    type: RS232,
    index: 1,
    configuration: {
        address: 0.0.0.0,
        dest_index: 1,
        enable: false,
        auto_stop: false
    },
    status: {
        state: STOPPED
    },
    source: {
        ref_class: NODE,
        ref_type: UART,
        ref_index: 1
    }
},
{
    type: RS232,
    index: 2,
    configuration: {
        address: 0.0.0.0,
        dest_index: 2,
        enable: false,
        auto_stop: false
    },
    status: {
        state: STOPPED
    },
    source: {
        ref_class: NODE,
        ref_type: UART,
        ref_index: 2
    }
},
{
    type: RS232,
    index: 3,
    configuration: {
        address: 0.0.0.0,
        dest_index: 3,
        enable: false,
        auto_stop: false
    },
    status: {
        state: STOPPED
    },

```

```

        source: {
            ref_class: NODE,
            ref_type: UART,
            ref_index: 3
        }
    },
    {
        type: INFRARED,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: INFRARED_DECODER,
            ref_index: 0
        }
    },
    {
        type: CEC,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: CEC,
            ref_index: 0
        }
    },
    {
        type: CEC,
        index: 1,
        configuration: {
            address: 0.0.0.0,
            enable: false,
            auto_stop: false
        },
        status: {
            state: STOPPED
        },
        source: {
            ref_class: NODE,
            ref_type: CEC,
            ref_index: 1
        }
    }
}

```

```

    },
    {
      type: USB_HID,
      index: 0,
      configuration: {
        address: 10.17.252.135,
        enable: true,
        auto_stop: false
      },
      status: {
        state: STREAMING
      },
      source: {
        ref_class: NODE,
        ref_type: USB_HID,
        ref_index: 0
      }
    }
  ],
  subscriptions: [
    {
      type: HDMI,
      index: 0,
      configuration: {
        address: 224.1.1.23,
        enable: true
      },
      status: {
        state: STREAMING
      }
    },
    {
      type: HDMI,
      index: 1,
      configuration: {
        address: 0.0.0.0,
        enable: false
      },
      status: {
        state: STOPPED
      }
    },
    {
      type: HDMI,
      index: 2,
      configuration: {
        address: 0.0.0.0,
        enable: false
      },
      status: {
        state: STOPPED
      }
    },
    {
      type: HDMI,
      index: 3,

```

```

        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 4,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 5,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 6,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 7,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 8,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {

```

```

        state: STOPPED
    },
    {
        type: HDMI,
        index: 9,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 10,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 11,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 12,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 13,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    }
}

```

```

    }
  },
  {
    type: HDMI,
    index: 14,
    configuration: {
      address: 0.0.0.0,
      enable: false
    },
    status: {
      state: STOPPED
    }
  },
  {
    type: HDMI,
    index: 15,
    configuration: {
      address: 0.0.0.0,
      enable: false
    },
    status: {
      state: STOPPED
    }
  },
  {
    type: HDMI,
    index: 16,
    configuration: {
      address: 0.0.0.0,
      enable: false
    },
    status: {
      state: STOPPED
    }
  },
  {
    type: HDMI,
    index: 17,
    configuration: {
      address: 0.0.0.0,
      enable: false
    },
    status: {
      state: STOPPED
    }
  },
  {
    type: HDMI,
    index: 18,
    configuration: {
      address: 0.0.0.0,
      enable: false
    },
    status: {
      state: STOPPED
    }
  }
}

```

```

    },
    {
      type: HDMI,
      index: 19,
      configuration: {
        address: 0.0.0.0,
        enable: false
      },
      status: {
        state: STOPPED
      }
    },
    {
      type: HDMI,
      index: 20,
      configuration: {
        address: 0.0.0.0,
        enable: false
      },
      status: {
        state: STOPPED
      }
    },
    {
      type: HDMI,
      index: 21,
      configuration: {
        address: 0.0.0.0,
        enable: false
      },
      status: {
        state: STOPPED
      }
    },
    {
      type: HDMI,
      index: 22,
      configuration: {
        address: 0.0.0.0,
        enable: false
      },
      status: {
        state: STOPPED
      }
    },
    {
      type: HDMI,
      index: 23,
      configuration: {
        address: 0.0.0.0,
        enable: false
      },
      status: {
        state: STOPPED
      }
    }
  ],

```

```

{
  type: HDMI,
  index: 24,
  configuration: {
    address: 0.0.0.0,
    enable: false
  },
  status: {
    state: STOPPED
  }
},
{
  type: HDMI,
  index: 25,
  configuration: {
    address: 0.0.0.0,
    enable: false
  },
  status: {
    state: STOPPED
  }
},
{
  type: HDMI,
  index: 26,
  configuration: {
    address: 0.0.0.0,
    enable: false
  },
  status: {
    state: STOPPED
  }
},
{
  type: HDMI,
  index: 27,
  configuration: {
    address: 0.0.0.0,
    enable: false
  },
  status: {
    state: STOPPED
  }
},
{
  type: HDMI,
  index: 28,
  configuration: {
    address: 0.0.0.0,
    enable: false
  },
  status: {
    state: STOPPED
  }
},
{

```

```

        type: HDMI,
        index: 29,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 30,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI,
        index: 31,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: HDMI_AUDIO,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false,
            mute_msec: 1500
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: STEREO_AUDIO,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    }
},
{

```

```

        type: MULTICH_AUDIO,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: false
        },
        status: {
            state: STOPPED
        }
    },
    {
        type: RS232,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    },
    {
        type: RS232,
        index: 1,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    },
    {
        type: RS232,
        index: 2,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    },
    {
        type: RS232,
        index: 3,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    },
    {
        type: INFRARED,

```

```

        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    },
    {
        type: USB_HID,
        index: 0,
        configuration: {
            address: 0.0.0.0,
            enable: true
        },
        status: {
            state: STREAMING
        }
    }
],
nodes: [
    {
        type: STEREO_AUDIO_INPUT_OUTPUT,
        index: 0,
        configuration: {
            direction: INPUT
        },
        status: {},
        inputs: [
            {
                name: main,
                index: 0,
                configuration: {
                    source: {
                        value: 2,
                        choices: [
                            {
                                value: 2,
                                description: Network HDMI
                            }
                        ]
                    },
                    details: {
                        ref_class: SUBSCRIPTION,
                        ref_type: HDMI_AUDIO,
                        ref_index: 0
                    }
                },
                value: 3,
                description: Network Stereo
            },
            {
                details: {
                    ref_class: SUBSCRIPTION,
                    ref_type: STEREO_AUDIO,
                    ref_index: 0
                }
            }
        ]
    }
]

```

Downmix Audio,

Audio,

```

    },
    {
        value: 8,
        description: Network Multi

Channel Audio,

        details: {
            ref_class: SUBSCRIPTION,
            ref_type: MULTICH_AUDIO,
            ref_index: 0
        }
    },
    {
        value: 9,
        description: Local Multi

Channel Audio,

        details: {
            ref_class: NODE,
            ref_type:

MULTICH_AUDIO_INPUT,

            ref_index: 0
        }
    },
    {
        value: 12,
        description: Local HDMI

Downmix Audio,

        details: {
            ref_class: NODE,
            ref_type: HDMI_DECODER,
            ref_index: 0
        }
    }
]
},
status: {
    source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI_AUDIO,
        ref_index: 0
    }
}
}
}
],
{
    type: CEC,
    index: 0,
    configuration: {
        p2p_mode: PASSTHROUGH
    },
    status: {},
    inputs: []
},
{
    type: CEC,

```

```

        index: 1,
        configuration: {
            p2p_mode: PASSTHROUGH
        },
        status: {},
        inputs: []
    },
    {
        type: COLOR_GENERATOR,
        index: 0,
        configuration: {
            enable: false,
            color: 000000,
            force_output: false
        },
        status: {},
        inputs: []
    },
    {
        type: FRAME_BUFFER,
        index: 0,
        configuration: {
            display_mode: FAST_SWITCHED,
            width: 1280,
            height: 720,
            horiz_offset: 0,
            vert_offset: 0,
            frames_per_second: 60,
            keep_width: 3840,
            keep_height: 2160,
            viewport_horiz: 0,
            viewport_vert: 0,
            viewport_width: 1280,
            viewport_height: 720,
            aspect_ratio_mode: KEEP,
            has_auto_wall_details: false,
            wall_name: ,
            freeze: false
        },
        status: {
            source_video: {
                width: 3840,
                height: 2160,
                frames_per_second: 60,
                color_space: RGB,
                bits_per_pixel: 8,
                scan_mode: PROGRESSIVE
            }
        },
        inputs: [
            {
                name: main,
                index: 0,
                configuration: {},
                status: {
                    source: {

```

```

        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 0
    }
}
},
{
    name: multiview,
    index: 0,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 0
        }
    }
},
{
    name: multiview,
    index: 1,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 1
        }
    }
},
{
    name: multiview,
    index: 2,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 2
        }
    }
},
{
    name: multiview,
    index: 3,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 3
        }
    }
},
{
    name: multiview,

```

```

        index: 4,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 4
            }
        }
    },
    {
        name: multiview,
        index: 5,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 5
            }
        }
    },
    {
        name: multiview,
        index: 6,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 6
            }
        }
    },
    {
        name: multiview,
        index: 7,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 7
            }
        }
    },
    {
        name: multiview,
        index: 8,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 8
            }
        }
    }
}

```

```

    }
  },
  {
    name: multiview,
    index: 9,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 9
      }
    }
  },
  {
    name: multiview,
    index: 10,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 10
      }
    }
  },
  {
    name: multiview,
    index: 11,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 11
      }
    }
  },
  {
    name: multiview,
    index: 12,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 12
      }
    }
  },
  {
    name: multiview,
    index: 13,
    configuration: {},
    status: {
      source: {

```

```

        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 13
    }
}
},
{
    name: multiview,
    index: 14,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 14
        }
    }
},
{
    name: multiview,
    index: 15,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 15
        }
    }
},
{
    name: multiview,
    index: 16,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 16
        }
    }
},
{
    name: multiview,
    index: 17,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 17
        }
    }
},
{
    name: multiview,

```

```

        index: 18,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 18
            }
        }
    },
    {
        name: multiview,
        index: 19,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 19
            }
        }
    },
    {
        name: multiview,
        index: 20,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 20
            }
        }
    },
    {
        name: multiview,
        index: 21,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 21
            }
        }
    },
    {
        name: multiview,
        index: 22,
        configuration: {},
        status: {
            source: {
                ref_class: SUBSCRIPTION,
                ref_type: HDMI,
                ref_index: 22
            }
        }
    }
}

```

```

    }
  },
  {
    name: multiview,
    index: 23,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 23
      }
    }
  },
  {
    name: multiview,
    index: 24,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 24
      }
    }
  },
  {
    name: multiview,
    index: 25,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 25
      }
    }
  },
  {
    name: multiview,
    index: 26,
    configuration: {},
    status: {
      source: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 26
      }
    }
  },
  {
    name: multiview,
    index: 27,
    configuration: {},
    status: {
      source: {

```

```

        ref_class: SUBSCRIPTION,
        ref_type: HDMI,
        ref_index: 27
    }
}
},
{
    name: multiview,
    index: 28,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 28
        }
    }
},
{
    name: multiview,
    index: 29,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 29
        }
    }
},
{
    name: multiview,
    index: 30,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 30
        }
    }
},
{
    name: multiview,
    index: 31,
    configuration: {},
    status: {
        source: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI,
            ref_index: 31
        }
    }
}
}
],
},

```

```

{
  type: GPIO,
  index: 0,
  configuration: {
    tristate: 1,
    cfg_val: 0
  },
  status: {
    stat_val: 0
  },
  inputs: []
},
{
  type: GPIO,
  index: 1,
  configuration: {
    tristate: 1,
    cfg_val: 0
  },
  status: {
    stat_val: 0
  },
  inputs: []
},
{
  type: GPIO,
  index: 2,
  configuration: {
    tristate: 1,
    cfg_val: 0
  },
  status: {
    stat_val: 0
  },
  inputs: []
},
{
  type: GPIO,
  index: 3,
  configuration: {
    tristate: 1,
    cfg_val: 0
  },
  status: {
    stat_val: 0
  },
  inputs: []
},
{
  type: GPIO,
  index: 4,
  configuration: {
    tristate: 1,
    cfg_val: 0
  },
  status: {

```

```

        stat_val: 0
    },
    inputs: []
},
{
    type: GPIO,
    index: 5,
    configuration: {
        tristate: 1,
        cfg_val: 0
    },
    status: {
        stat_val: 0
    },
    inputs: []
},
{
    type: GPIO,
    index: 6,
    configuration: {
        tristate: 1,
        cfg_val: 0
    },
    status: {
        stat_val: 0
    },
    inputs: []
},
{
    type: GPIO,
    index: 7,
    configuration: {
        tristate: 1,
        cfg_val: 0
    },
    status: {
        stat_val: 0
    },
    inputs: []
},
{
    type: GPIO,
    index: 8,
    configuration: {
        tristate: 1,
        cfg_val: 0
    },
    status: {
        stat_val: 0
    },
    inputs: []
},
{
    type: GPIO,
    index: 9,
    configuration: {

```

```

        tristate: 1,
        cfg_val: 0
    },
    status: {
        stat_val: 0
    },
    inputs: []
},
{
    type: GPIO,
    index: 10,
    configuration: {
        tristate: 1,
        cfg_val: 0
    },
    status: {
        stat_val: 0
    },
    inputs: []
},
{
    type: GPIO,
    index: 11,
    configuration: {
        tristate: 1,
        cfg_val: 0
    },
    status: {
        stat_val: 0
    },
    inputs: []
},
{
    type: HDMI_DECODER,
    index: 0,
    configuration: {
        edid:
00ffffffffffff000614000156524c420816010380341d780a01c1a057479827124c4c2108008
140010101010101010101010101010101023a801871382d40582c460040846300001e08e80030f2
705a80b0588a00ba892100001e000000fc00426c756552697665720a202020000000fd0018550
e853c000a202020202020017c02034373580102030405111213141f2021220607905d5e5f6263
646061230d0707830f00006d030c001000383c20406801020367d85dc401788003e40f0000c0e
200cf011d00bc52d01e20b828554040846300001e023a80d072382d40102c458040846300001e
023a801871382d40582c450040846300001e00000000000003e,
        hdcp_support_enable: true,
        hdcp_22_support_disable: false,
        default_quantization_level_override: DISABLED
    },
    status: {
        video: {
            width: 1920,
            height: 1080,
            frames_per_second: 60,
            color_space: RGB,
            bits_per_pixel: 8,
            scan_mode: PROGRESSIVE

```

```

    },
    has_video_details: true,
    video_details: {
        frame_rate: 60,
        pixel_clock: 148495683,
        total_width: 2200,
        total_height: 1125,
        hsync_width: 44,
        hsync_front_porch: 88,
        hsync_negative: false,
        vsync_width: 6,
        vsync_front_porch: 4,
        vsync_negative: false,
        vic: 16,
        has_hdmi_vic: false,
        hdmi_vic: 0,
        picture_aspect: ASPECT_16_9,
        has_active_format: false,
        active_format: 0,
        it_content_type: GRAPHICS,
        scan_information: NO_DATA,
        colorimetry: BT709,
        rgb_range: DEFAULT,
        ycc_range: LIMITED,
        has_hdr: false,
        hdr: SDR
    },
    has_audio_details: true,
    audio_details: {
        sampling_frequency: 48000,
        number_of_channels: 2
    },
    has_audio_details_extended: true,
    audio_details_extended: {
        channel_allocation: 0,
        sample_size: 16,
        level_shift_value: 0,
        lfe_playback_level: 0,
        audio_encoding_type: LPCM,
        is_downmix_inhibited: false
    },
    hdcp_protected: false,
    hdcp_version: NONE,
    hdmi_2_0_support: true,
    source_stable: true,
    audio_downmix_support: true
},
inputs: [
    {
        name: main,
        index: 0,
        configuration: {},
        status: {
            source: {
                ref_class: NODE,
                ref_type: VIDEO_INPUT,
            }
        }
    }
]

```

```

        ref_index: 0
    }
}
]
},
{
    type: HDMI_ENCODER,
    index: 0,
    configuration: {
        hdcp_output_mode: FOLLOW_SINK_1,
        hdmi_tx_5v: AUTO
    },
    status: {
        video: {
            width: 1280,
            height: 720,
            frames_per_second: 60,
            color_space: RGB,
            bits_per_pixel: 8,
            scan_mode: PROGRESSIVE
        },
        has_video_details: true,
        video_details: {
            frame_rate: 60,
            pixel_clock: 74250000,
            total_width: 1650,
            total_height: 750,
            hsync_width: 40,
            hsync_front_porch: 110,
            hsync_negative: false,
            vsync_width: 5,
            vsync_front_porch: 5,
            vsync_negative: false,
            vic: 4,
            has_hdmi_vic: false,
            hdmi_vic: 0,
            picture_aspect: ASPECT_16_9,
            has_active_format: false,
            active_format: 0,
            it_content_type: NO_DATA,
            scan_information: NO_DATA,
            colorimetry: RGB,
            rgb_range: LIMITED,
            ycc_range: LIMITED,
            has_hdr: false,
            hdr: SDR
        },
        has_audio_details: true,
        audio_details: {
            sampling_frequency: 44100,
            number_of_channels: 2
        },
        has_audio_details_extended: true,
        audio_details_extended: {
            channel_allocation: 0,

```

```

        sample_size: 0,
        level_shift_value: 0,
        lfe_playback_level: 0,
        audio_encoding_type: LPCM,
        is_downmix_inhibited: false
    },
    hdcp_protected: false,
    hdcp_version: NONE,
    hdmi_2_0_support: true,
    source_stable: true
},
inputs: [
    {
        name: main,
        index: 0,
        configuration: {
            source: {
                value: 1,
                choices: [
                    {
                        value: 0,
                        description: Genlock Mode,
                        details: {
                            ref_class: SUBSCRIPTION,
                            ref_type: HDMI,
                            ref_index: 0
                        }
                    },
                    {
                        value: 1,
                        description: Frame Buffer
                        details: {
                            ref_class: NODE,
                            ref_type: FRAME_BUFFER,
                            ref_index: 0
                        }
                    }
                ]
            }
        }
    },
    {
        status: {
            source: {
                ref_class: NODE,
                ref_type: FRAME_BUFFER,
                ref_index: 0
            }
        }
    }
],
{
    name: audio,
    index: 0,
    configuration: {
        source: {
            value: 2,
            choices: [

```

Mode,

Audio Aux (Video Genlock),

Audio,

Downmix Audio,

Audio,

Channel Audio,

Channel Audio,

```
{
  value: 2,
  description: Network HDMI

  details: {
    ref_class: SUBSCRIPTION,
    ref_type: HDMI_AUDIO,
    ref_index: 0
  }
},
{
  value: 3,
  description: Network Stereo

  details: {
    ref_class: SUBSCRIPTION,
    ref_type: STEREO_AUDIO,
    ref_index: 0
  }
},
{
  value: 6,
  description: Network HDMI

  details: {
    ref_class: SUBSCRIPTION,
    ref_type: HDMI_AUDIO,
    ref_index: 0
  }
},
{
  value: 7,
  description: Network HDMI

  details: {
    ref_class: SUBSCRIPTION,
    ref_type: HDMI_AUDIO,
    ref_index: 0
  }
},
{
  value: 8,
  description: Network Multi

  details: {
    ref_class: SUBSCRIPTION,
    ref_type: MULTICH_AUDIO,
    ref_index: 0
  }
},
{
  value: 9,
  description: Local Multi

  details: {
    ref_class: NODE,
```

```

MULTICH_AUDIO_INPUT,
ref_type:
ref_index: 0
}
}
]
}
},
status: {
source: {
ref_class: SUBSCRIPTION,
ref_type: HDMI_AUDIO,
ref_index: 0
}
}
}
]
},
{
type: HDMI_MONITOR,
index: 0,
configuration: {},
status: {
connected: true,
edid:
00ffffffffffff00220e173401010101061b0103803c22782a2895a7554ea3260f5054a10800d
1c081c081809500a9c0a940b300d1004dd000a0f0703e803040350055502100001a000000fd00
283c87873c000a202020202020000000fc00485020454e5659203237730a20000000ff0036434
d373036304e52530a202001b7020335b14b6160101f04130312021101e2002b6c030c00100038
3c000000000067d85dc40178800681a00000101283cede30f0000565e00a0a0a029503020350
055502100001ae26800a0a0402e602030630055502100001c000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000f
},
inputs: [
{
name: main,
index: 0,
configuration: {},
status: {
source: {
ref_class: NODE,
ref_type: HDMI_ENCODER,
ref_index: 0
}
}
}
]
},
{
type: MULTICH_AUDIO_INPUT,
index: 0,
configuration: {
sampling_frequency: {
value: 48000,
choices: [
{

```

```

        value: 22050,
        description: 22.05kHz,
        details: null
    },
    {
        value: 24000,
        description: 24kHz,
        details: null
    },
    {
        value: 32000,
        description: 32kHz,
        details: null
    },
    {
        value: 44100,
        description: 44.1kHz,
        details: null
    },
    {
        value: 48000,
        description: 48kHz,
        details: null
    },
    {
        value: 88200,
        description: 88.2kHz,
        details: null
    },
    {
        value: 96000,
        description: 96kHz,
        details: null
    },
    {
        value: 176400,
        description: 176.4kHz,
        details: null
    },
    {
        value: 192000,
        description: 192kHz,
        details: null
    },
    {
        value: 768000,
        description: 768kHz,
        details: null
    }
    ]
},
number_of_channels: {
    value: 8,
    choices: [
        {
            value: 2,

```

```

        description: 2 Channels,
        details: null
    },
    {
        value: 3,
        description: 3 Channels,
        details: null
    },
    {
        value: 4,
        description: 4 Channels,
        details: null
    },
    {
        value: 5,
        description: 5 Channels,
        details: null
    },
    {
        value: 6,
        description: 6 Channels,
        details: null
    },
    {
        value: 7,
        description: 7 Channels,
        details: null
    },
    {
        value: 8,
        description: 8 Channels,
        details: null
    }
]
},
channel_allocation: {
    value: 0,
    choices: [
        {
            value: 0,
            description: 2 Channels (FL-FR),
            details: null
        },
        {
            value: 1,
            description: 2.1 Channels (FL-FR-
LFE),
            details: null
        },
        {
            value: 2,
            description: 3 Channels (FL-FR-X-FC),
            details: null
        },
        {
            value: 3,

```

```

FC) ,
    description: 3.1 Channels (FL-FR-LFE-
    details: null
},
{
    value: 4,
    description: 3 Channels (FL-FR-X-X-
RC) ,
    details: null
},
{
    value: 5,
    description: 3.1 Channels (FL-FR-LFE-
X-RC) ,
    details: null
},
{
    value: 6,
    description: 4 Channels (FL-FR-X-FC-
RC) ,
    details: null
},
{
    value: 7,
    description: 4.1 Channels (FL-FR-LFE-
FC-RC) ,
    details: null
},
{
    value: 8,
    description: 4 Channels (FL-FR-X-X-
RL-RR) ,
    details: null
},
{
    value: 9,
    description: 4.1 Channels (FL-FR-LFE-
X-RL-RR) ,
    details: null
},
{
    value: 10,
    description: 5 Channels (FL-FR-X-FC-
RL-RR) ,
    details: null
},
{
    value: 11,
    description: 5.1 Channels (FL-FR-LFE-
FC-RL-RR) ,
    details: null
},
{
    value: 12,
    description: 5 Channels (FL-FR-X-X-
RL-RR-RC) ,

```

```

        details: null
    },
    {
        value: 13,
        description: 5.1 Channels (FL-FR-LFE-
X-RL-RR-RC) ,
        details: null
    },
    {
        value: 14,
        description: 6 Channels (FL-FR-X-FC-
RL-RR-RC) ,
        details: null
    },
    {
        value: 15,
        description: 6.1 Channels (FL-FR-LFE-
FC-RL-RR-RC) ,
        details: null
    },
    {
        value: 16,
        description: 6 Channels (FL-FR-X-X-
RL-RR-RLC-RRC) ,
        details: null
    },
    {
        value: 17,
        description: 6.1 Channels (FL-FR-LFE-
X-RL-RR-RLC-RRC) ,
        details: null
    },
    {
        value: 18,
        description: 7 Channels (FL-FR-X-FC-
RL-RR-RLC-RRC) ,
        details: null
    },
    {
        value: 19,
        description: 7.1 Channels (FL-FR-LFE-
FC-RL-RR-RLC-RRC) ,
        details: null
    },
    {
        value: 20,
        description: 4 Channels (FL-FR-X-X-X-
X-FLC-FRC) ,
        details: null
    },
    {
        value: 21,
        description: 4.1 Channels (FL-FR-LFE-
X-X-X-FLC-FRC) ,
        details: null
    },
    },

```

```

X-X-FLC-FRC),
    {
        value: 22,
        description: 5 Channels (FL-FR-X-FC-
    },
    {
        value: 23,
        description: 5.1 Channels (FL-FR-LFE-
    },
    {
        value: 24,
        description: 5 Channels (FL-FR-X-X-
    },
    {
        value: 25,
        description: 5.1 Channels (FL-FR-LFE-
    },
    {
        value: 26,
        description: 6 Channels (FL-FR-X-FC-
    },
    {
        value: 27,
        description: 6.1 Channels (FL-FR-LFE-
    },
    {
        value: 28,
        description: 6 Channels (FL-FR-X-X-
    },
    {
        value: 29,
        description: 6.1 Channels (FL-FR-LFE-
    },
    {
        value: 30,
        description: 7 Channels (FL-FR-X-FC-
    },
    {
        value: 31,

```

```

description: 7.1 Channels (FL-FR-LFE-
FC-RL-RR-FLC-FRC) ,
details: null
},
{
value: 32,
description: 6 Channels (X-FCH-RR-RL-
FC-X-FR-FL) ,
details: null
},
{
value: 33,
description: 6.1 Channels (X-FCH-RR-
RL-FC-LFE-FR-FL) ,
details: null
},
{
value: 34,
description: 6 Channels (TC-X-RR-RL-
FC-X-FR-FL) ,
details: null
},
{
value: 35,
description: 6.1 Channels (TC-X-RR-
RL-FC-LFE-FR-FL) ,
details: null
},
{
value: 36,
description: 6 Channels (FRH-FLH-RR-
RL-X-X-FR-FL) ,
details: null
},
{
value: 37,
description: 6.1 Channels (FRH-FLH-
RR-RL-X-LFE-FR-FL) ,
details: null
},
{
value: 38,
description: 6 Channels (FRW-FLW-RR-
RL-X-X-FR-FL) ,
details: null
},
{
value: 39,
description: 6.1 Channels (FRW-FLW-
RR-RL-X-LFE-FR-FL) ,
details: null
},
{
value: 40,
description: 7 Channels (TC-RC-RR-RL-
FC-X-FR-FL) ,

```

```

        details: null
    },
    {
        value: 41,
        description: 7.1 Channels (TC-RC-RR-
RL-FC-LFE-FR-FL) ,
        details: null
    },
    {
        value: 42,
        description: 7 Channels (FCH-RC-RR-
RL-FC-X-FR-FL) ,
        details: null
    },
    {
        value: 43,
        description: 7.1 Channels (FCH-RC-RR-
RL-FC-LFE-FR-FL) ,
        details: null
    },
    {
        value: 44,
        description: 7 Channels (TC-FCH-RR-
RL-FC-X-FR-FL) ,
        details: null
    },
    {
        value: 45,
        description: 7.1 Channels (TC-FCH-RR-
RL-FC-LFE-FR-FL) ,
        details: null
    },
    {
        value: 46,
        description: 7 Channels (FRH-FLH-RR-
RL-FC-X-FR-FL) ,
        details: null
    },
    {
        value: 47,
        description: 7.1 Channels (FRH-FLH-
RR-RL-FC-LFE-FR-FL) ,
        details: null
    },
    {
        value: 48,
        description: 7 Channels (FRW-FLW-RR-
RL-FC-X-FR-FL) ,
        details: null
    },
    {
        value: 49,
        description: 7.1 Channels (FRW-FLW-
RR-RL-FC-LFE-FR-FL) ,
        details: null
    }
}

```

```

    ],
    },
    audio_encoding_type: LPCM,
    mute_audio: false
  },
  status: {
    data_lanes: 4,
    sampling_frequency: 0
  },
  inputs: []
},
{
  type: MULTICH_AUDIO_OUTPUT,
  index: 0,
  configuration: {},
  status: {
    data_lanes: 4
  },
  inputs: [
    {
      name: main,
      index: 0,
      configuration: {
        source: {
          value: 8,
          choices: [
            {
              value: 3,
              description: Network Stereo
            }
          ],
          details: {
            ref_class: SUBSCRIPTION,
            ref_type: STEREO_AUDIO,
            ref_index: 0
          }
        },
        {
          value: 6,
          description: Network HDMI
        },
        {
          value: 7,
          description: Network HDMI
        },
        {
          value: 7,
          description: Network HDMI
        },
        {
          details: {
            ref_class: SUBSCRIPTION,
            ref_type: HDMI_AUDIO,
            ref_index: 0
          }
        }
      ],
      details: {
        ref_class: SUBSCRIPTION,
        ref_type: HDMI_AUDIO,
        ref_index: 0
      }
    }
  ],
  details: {
    ref_class: SUBSCRIPTION,
    ref_type: HDMI_AUDIO,
    ref_index: 0
  }
},

```

Audio,

Downmix Audio,

Audio,

Channel Audio,

Audio,

STEREO_AUDIO_INPUT_OUTPUT,

Downmix Audio,

Audio,

```
{
  value: 8,
  description: Network Multi
  details: {
    ref_class: SUBSCRIPTION,
    ref_type: MULTICH_AUDIO,
    ref_index: 0
  }
},
{
  value: 10,
  description: Local Stereo
  details: {
    ref_class: NODE,
    ref_type:
    ref_index: 0
  }
},
{
  value: 12,
  description: Local HDMI
  details: {
    ref_class: NODE,
    ref_type: HDMI_DECODER,
    ref_index: 0
  }
},
{
  value: 11,
  description: Local HDMI
  details: {
    ref_class: NODE,
    ref_type: HDMI_DECODER,
    ref_index: 0
  }
}
]
},
status: {
  source: {
    ref_class: SUBSCRIPTION,
    ref_type: MULTICH_AUDIO,
    ref_index: 0
  }
}
}
]
},
{
  type: INFRARED_ENCODER,
```

```

        index: 0,
        configuration: {},
        status: {
            data_length_max: 1032
        },
        inputs: [
            {
                name: network,
                index: 0,
                configuration: {},
                status: {
                    source: {
                        ref_class: SUBSCRIPTION,
                        ref_type: INFRARED,
                        ref_index: 0
                    }
                }
            }
        ]
    },
    {
        type: INFRARED_DECODER,
        index: 0,
        configuration: {},
        status: {},
        inputs: []
    },
    {
        type: LED,
        index: 0,
        configuration: {
            function: {
                value: 0,
                choices: [
                    {
                        value: 0,
                        description: (Unknown),
                        details: UNKNOWN
                    },
                    {
                        value: 1,
                        description: Blink LED,
                        details: BLINK
                    },
                    {
                        value: 8,
                        description: Default functionality,
                        details: DEFAULT
                    }
                ]
            }
        }
    },
    status: {},
    inputs: []
},
{

```

```

    type: NETWORK_INTERFACE,
    index: 0,
    configuration: {
      hostname: TR_W1BL,
      ip: {
        address: 10.17.252.109,
        mode: DHCP,
        mask: 255.255.255.0,
        gateway: 10.17.252.254
      },
      dhcp_option_43: {
        control_server_addresses: []
      }
    },
    status: {
      mac_address: 0016c04c40d9,
      ip: {
        address: 10.17.252.109
      }
    },
    inputs: []
  },
  {
    type: NETWORK_PORT,
    index: 0,
    configuration: {
      speed: SPEED_10G
    },
    status: {
      active: true,
      speed: 10,
      supports_speed_10g: true,
      supports_speed_5g: false,
      supports_speed_2_5g: false,
      supports_auto_10g: false,
      supports_auto_5g: false
    },
    inputs: []
  },
  {
    type: NETWORK_SWITCH,
    index: 0,
    configuration: {
      gigabit_port_1_enable: true
    },
    status: {},
    inputs: []
  },
  {
    type: SCALER,
    index: 0,
    configuration: {
      width: 0,
      height: 0
    },
    status: {

```

```

        video: {
            width: 0,
            height: 0,
            frames_per_second: 60,
            color_space: RGB,
            bits_per_pixel: 8,
            scan_mode: PROGRESSIVE
        }
    },
    inputs: [
        {
            name: main,
            index: 0,
            configuration: {},
            status: {
                source: {
                    ref_class: NODE,
                    ref_type: HDMI_DECODER,
                    ref_index: 0
                }
            }
        }
    ]
},
{
    type: UART,
    index: 0,
    configuration: {
        baud_rate: 57600,
        data_bits: 8,
        stop_bits: 1,
        parity: NONE
    },
    status: {
        usage: EXPOSED
    },
    inputs: [
        {
            name: network,
            index: 0,
            configuration: {},
            status: {
                source: {
                    ref_class: SUBSCRIPTION,
                    ref_type: RS232,
                    ref_index: 0
                }
            }
        }
    ]
},
{
    type: UART,
    index: 1,
    configuration: {
        baud_rate: 57600,

```

```

        data_bits: 8,
        stop_bits: 1,
        parity: NONE
    },
    status: {
        usage: EXPOSED
    },
    inputs: [
        {
            name: network,
            index: 1,
            configuration: {},
            status: {
                source: {
                    ref_class: SUBSCRIPTION,
                    ref_type: RS232,
                    ref_index: 1
                }
            }
        }
    ]
},
{
    type: UART,
    index: 2,
    configuration: {
        baud_rate: 57600,
        data_bits: 8,
        stop_bits: 1,
        parity: NONE
    },
    status: {
        usage: EXPOSED
    },
    inputs: [
        {
            name: network,
            index: 2,
            configuration: {},
            status: {
                source: {
                    ref_class: SUBSCRIPTION,
                    ref_type: RS232,
                    ref_index: 2
                }
            }
        }
    ]
},
{
    type: UART,
    index: 3,
    configuration: {
        baud_rate: 57600,
        data_bits: 8,
        stop_bits: 1,

```

```

        parity: NONE
    },
    status: {
        usage: EXPOSED
    },
    inputs: [
        {
            name: network,
            index: 3,
            configuration: {},
            status: {
                source: {
                    ref_class: SUBSCRIPTION,
                    ref_type: RS232,
                    ref_index: 3
                }
            }
        }
    ]
},
{
    type: USB_HID,
    index: 0,
    configuration: {
        role: LOCAL,
        peers: [
            {
                pairing_type: DEVICE_ID,
                address: ,
                device_id: 0016c04c46d8
            }
        ]
    },
    status: {
        usb_devices: [
            {
                status: CONNECTED
            },
            {
                status: CONNECTED
            },
            {
                status: CONNECTED
            }
        ],
        usb_host: {
            status: CONNECTED
        },
        role: LOCAL,
        peers: [
            {
                linked: true
            }
        ]
    },
    inputs: []
}

```

```

    },
    {
      type: USB_EXTENDER,
      index: 0,
      configuration: {
        role: LOCAL,
        program_mode: NONE,
        rs232_port: 0,
        peers: [ {
          avp_attached: {
            device_id: 001122334455,
            device_name: AVP_001122334455,
          }
          device_id: 66778899AABB,
          device_name: USB_66778899AABB,
          mac_address: 66778899AABB,
        } ],
        device_filtering: {
          block_iso_transfers: false,
          class_filtering: ALLOW_ALL,
          allow_api_commands: false,
          allow_i2c_control: false,
        }
        remote_vbus_ctrl: AUTO,
        msa_enable: {
          bot_protocol: true,
          uas_protocol: true,
        }
      },
      status: {
        chip_present: true,
        device_id: 0016c0355739,
        device_name: USB_0016c0355739,
        mac_address: 0016c0355739,
        chipset_type: UEX1000,
        reachable: true,
        revision: 1.0.0.0,
        ip_mode: DHCP,
        ip_address: 10.17.252.101,
        link_status_support: true,
        sui_enabled: false,
        peers: [ {
          linked: true
        } ],
        role: LOCAL
      },
      inputs: []
    },
    {
      type: VIDEO_COMPRESSOR,
      index: 0,
      configuration: {
        always_compress: false,
        version: 1
      },
      status: {

```

```

        version_support: 2
    },
    inputs: []
},
{
    type: VIDEO_DECOMPRESSOR,
    index: 0,
    configuration: {
        version: 1
    },
    status: {
        version_support: 2
    },
    inputs: []
},
{
    type: VIDEO_INPUT,
    index: 0,
    configuration: {},
    status: {},
    inputs: []
}
],
usb_extenders: [
{
    device_id: 0016c0355739,
    device_name: USB_0016c0355739,
    identity: {
        chipset_type: UEX1000,
        engine: USB2.0,
        vendor_id: 1,
        product_id: 212,
        firmware_comment: UEX 1.0.0.0_build986 uex_fpga,
        firmware_version: 1.0.0.0,
        chip_id: 0
    },
    capabilities: {
        local_role_supported: true,
        remote_role_supported: false,
        legacy_mode: false
    },
    configuration: {
        device_name: USB_0016c0355739,
        locate_mode: false,
        claimed: false,
        volatile_settings: false
    },
    status: {
        active: true,
        point_to_point: false,
        point_to_point_peer: ,
        update_in_progress: false,
        error_status: {
            has_error_code: false,
            error_code: 0
        }
    }
}
]

```

```

},
streams: [],
subscriptions: [],
nodes: [
  {
    type: NETWORK_INTERFACE,
    index: 0,
    configuration: {
      hostname: USB_0016c0355739,
      ip: {
        address: 10.17.252.115,
        mode: DHCP,
        mask: 255.255.255.0,
        gateway: 10.17.252.254
      }
    },
    status: {
      mac_address: 0016c0355739,
      ip: {
        address: 10.17.252.115
      }
    },
    inputs: []
  },
  {
    type: USB_EXTENDER,
    index: 0,
    configuration: {
      role: LOCAL,
      peers: [ {
        avp_attached: {
          device_id: 001122334455,
          device_name: AVP_001122334455,
        },
        device_id: 66778899AABB,
        device_name: USB_66778899AABB,
        mac_address: 66778899AABB,
      } ],
      device_filtering: {
        block_iso_transfers: false,
        class_filtering: ALLOW_ALL,
        allow_api_commands: false,
        allow_i2c_control: false,
      },
      remote_vbus_ctrl: AUTO,
      msa_enable: {
        bot_protocol: true,
        uas_protocol: true,
      }
    },
    status: {
      chip_present: true,
      device_id: 44b7d0cb86a3,
      device_name: USB_44b7d0cb86a3,
      mac_address: 44b7d0cb86a3,
      chipset_type: UEX1000,
    }
  }
]

```

```
        reachable: true,
        revision: 1.0.0.0,
        ip_mode: DHCP,
        ip_address: 10.17.252.115,
        link_status_support: true,
        sui_enabled: false,
        peers: [ {
            linked: true
        } ],
        role: LOCAL
    },
    inputs: []
}
}
}
    ],
    error: []
},
error: null
}
```

Semtech Confidential Information
Mark Li
ADV
02/03/2026

8 Revision History

Version	Date	Revision
1.3	December 2025	Updated to reflect API version 3.9.
1.2	February 2023	The <code>wall_name</code> argument and description added to section Video Format Arguments.
1.1	May 2022	Updated event types to reference new <code>MULTIVIEW_CHANGED</code> event, added in API version 3.4.
1.0	July 2021	Revision created for the SDVoE API release.

Semtech Confidential Information
Mark Li
ADV
02/03/2026



IMPORTANT NOTICE

Information relating to this product and the application or design described herein is believed to be reliable, however such information is provided as a guide only and Semtech assumes no liability for any errors in this document, or for the application or design described herein. Semtech reserves the right to make changes to the product or this document at any time without notice. Buyers should obtain the latest relevant information before placing orders and should verify that such information is current and complete. Semtech warrants performance of its products to the specifications applicable at the time of sale, and all sales are made in accordance with Semtech's standard terms and conditions of sale.

SEMTECH PRODUCTS ARE NOT DESIGNED, INTENDED, AUTHORIZED OR WARRANTED TO BE SUITABLE FOR USE IN LIFE-SUPPORT APPLICATIONS, DEVICES OR SYSTEMS, OR IN NUCLEAR APPLICATIONS IN WHICH THE FAILURE COULD BE REASONABLY EXPECTED TO RESULT IN PERSONAL INJURY, LOSS OF LIFE OR SEVERE PROPERTY OR ENVIRONMENTAL DAMAGE. INCLUSION OF SEMTECH PRODUCTS IN SUCH APPLICATIONS IS UNDERSTOOD TO BE UNDERTAKEN SOLELY AT THE CUSTOMER'S OWN RISK. Should a customer purchase or use Semtech products for any such unauthorized application, the customer shall indemnify and hold Semtech and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs damages and attorney fees which could arise.

Semtech, the Semtech logo and BlueRiver are registered trademarks of Semtech Corporation or its subsidiaries. All other trademarks and trade names mentioned may be marks and names of Semtech or their respective companies. Semtech reserves the right to make changes to, or discontinue any products described in this document without further notice. Semtech makes no warranty, representation, or guarantee, express or implied, regarding the suitability of its products for any particular purpose. All rights reserved.

© Semtech 2025

For more information contact: api@sdvov.org